

同济大学计算机系

计算机组成原理课程综合实验报告



学 号 2351579

姓 名 程浩然

专 业 计算机科学与技术

指导教师 张冬冬

目 录

1 实验内容	1
2 CPU 数据通路设计	2
2.1 每条指令的数据通路图	2
2.1.1 add 数据通路	2
2.1.2 sll 数据通路	3
2.1.3 jr 数据通路	4
2.1.4 add 数据通路	5
2.1.5 lw 数据通路	6
2.1.6 sw 数据通路	7
2.1.7 beq 数据通路	8
2.1.8 bne 数据通路	9
2.1.9 lui 数据通路	10
2.1.10 j 数据通路	11
2.1.11 jal 数据通路	12
2.2 整体数据通路	13
2.3 每条指令的要求	13
3 CPU 控制部件设计	15
3.1 操作时间表	15
3.2 各个指令的逻辑表达式	17
4 CPU 前仿真测试结果	18
4.1 modelsim 仿真	18
4.2 整体数据通路	18
4.3 仿真流程	20
5 CPU 后仿真	25
5.1 后仿真遇到的问题	25
5.2 后仿真效果图	28
5.3 CPU 下板结果	29
6 心得体会及建议	30
6.1 到底如何从零开始学习 cpu	30
6.2 对于实验文档的一些改进建议	30
7 附录：工程所有代码	31

1 实验内容

- 深入掌握 CPU 的构成及工作原理
- 设计 31 条指令的 CPU 的数据通路及控制器
- 使用 Verilog HD 语言设计实现 31 条指令的 CPU 下板运行

装

订

线

2 CPU 数据通路设计

2.1 每条指令的数据通路图

特别说明，为了减少工作量，提高文档阅读效率，我考虑将数据通路相同的指令合并展示。

2.1.1 add 数据通路

在本节，与 add 数据通路相同的指令包括，add，addu，sub，subu，and，or，xor，nor，slt，sltu，sllv，srlv，srav。数据通路如下

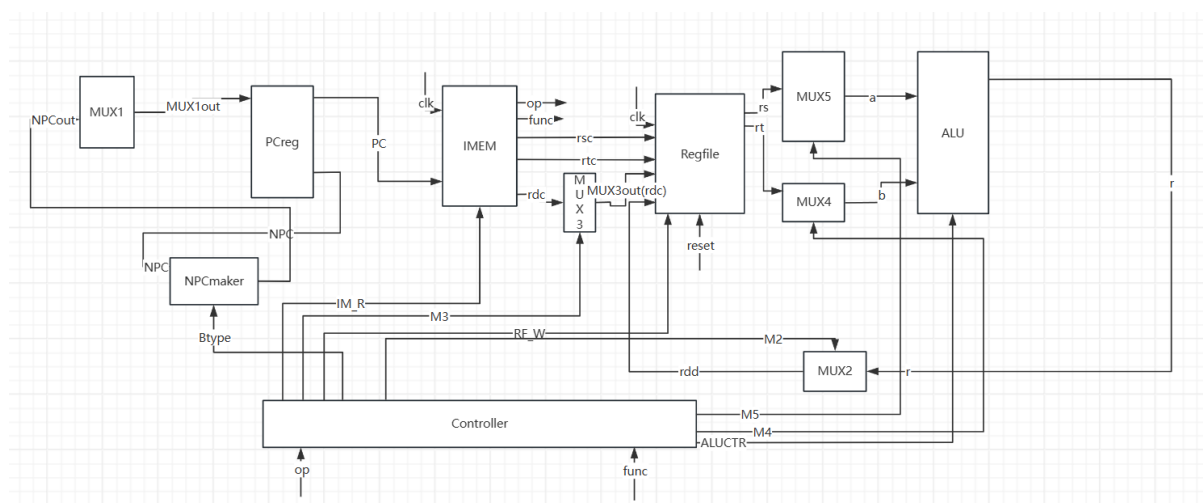


图 2.1 add 数据通路

在运行 add 的过程中，会执行如下动作：

在时钟的上升沿：

- PC->IMEM
- op->controller
- func->controller
- rsc->regfile
- rtc->regfile
- rdc->regfile
- rs->alu
- rt->alu
- alu 根据输入信号决定执行的指令
- r->rdd 更新 rd 的数据

在时钟的下降沿 npc-> PCreg

2.1.2 sll 数据通路

在本节当中，与 sll 数据通路相同的指令包括 srl, sra。数据通路如下：

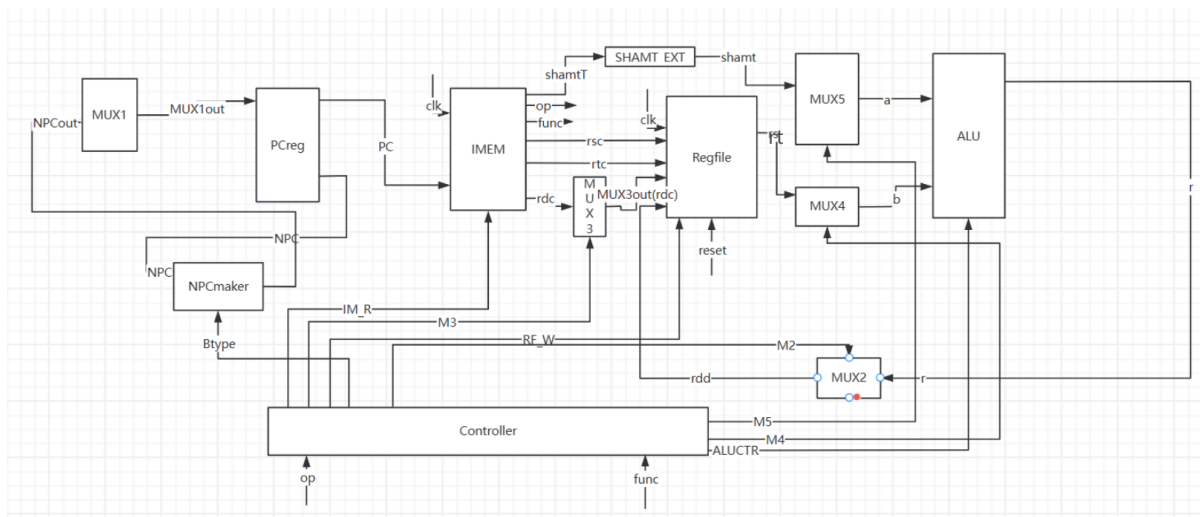


图 2.2 sll 数据通路

在运行 add 的过程中，会执行如下动作：

在时钟的上升沿：

- PC->IMEM
- op->controller
- func->controller
- rsc->regfile
- rdc->regfile
- shamtT->SHAMT_EXT
- shamt->alu
- rs->alu
- alu 根据输入信号决定执行的指令
- r->rdd 更新 rd 的数据

在时钟的下降沿 npc-> PCreg

2.1.3 jr 数据通路

在本节中，jr 拥有特别的数据通路

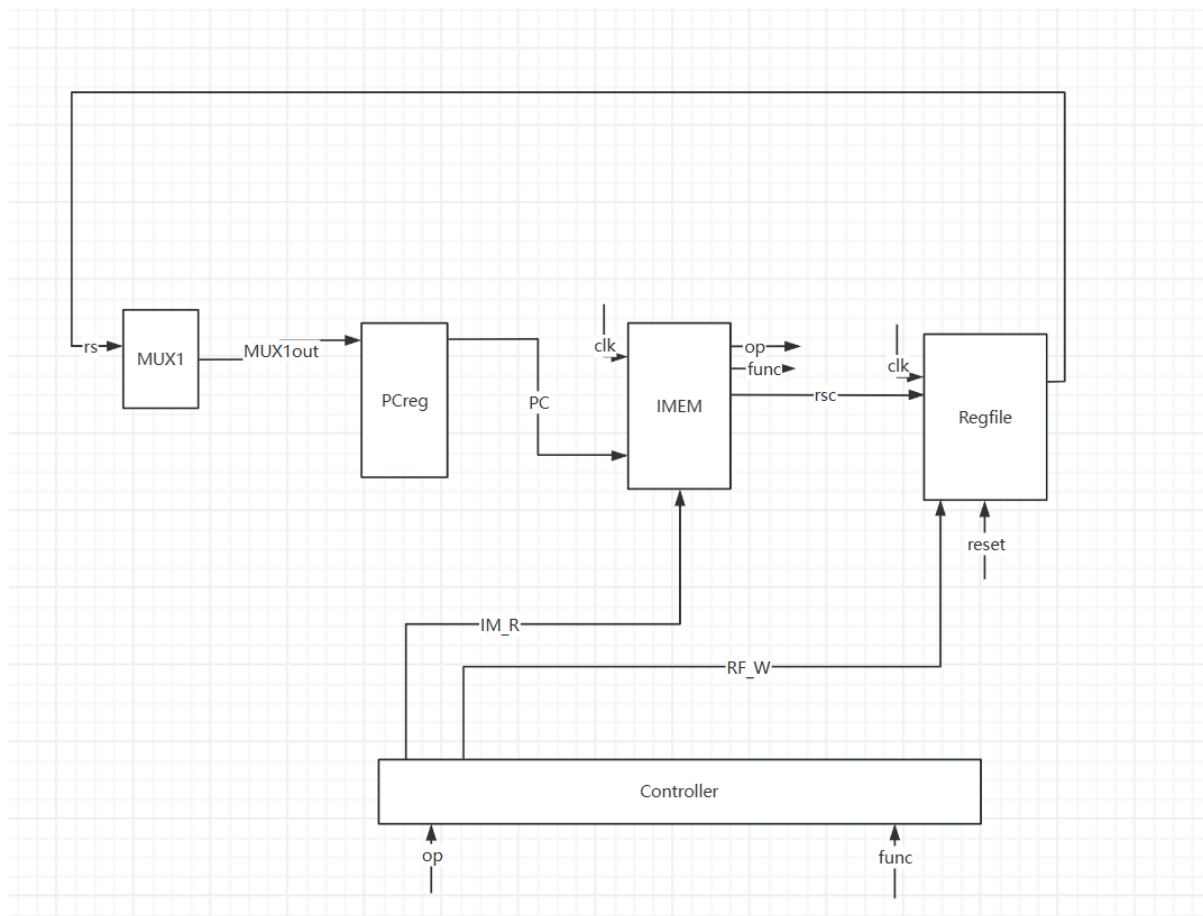


图 2.3 jr 数据通路

在执行 jr 指令的过程中，会执行如下操作：

在时钟的上升沿

- PC->IMEM
- op->controller
- func->controller
- rsc->regfile

在时钟的下降沿 rs->PCreg

2.1.4 add 数据通路

在本节，与 add 数据通路相同的指令包括，addi，addiu，addi，ori，xori。但是会产生一些无法在数据通路上的差异，对于立即数的扩展方式由一定的差别，控制立即数拓展方式的是信号 su，s 表示 signed，u 表示 unsigned，imdt_EXT 按照控制信号进行扩展。

需要特别说明的是，在 mips 指令架构下，addiu 作的是有符号拓展，让人感到意外。

数据通路如下：

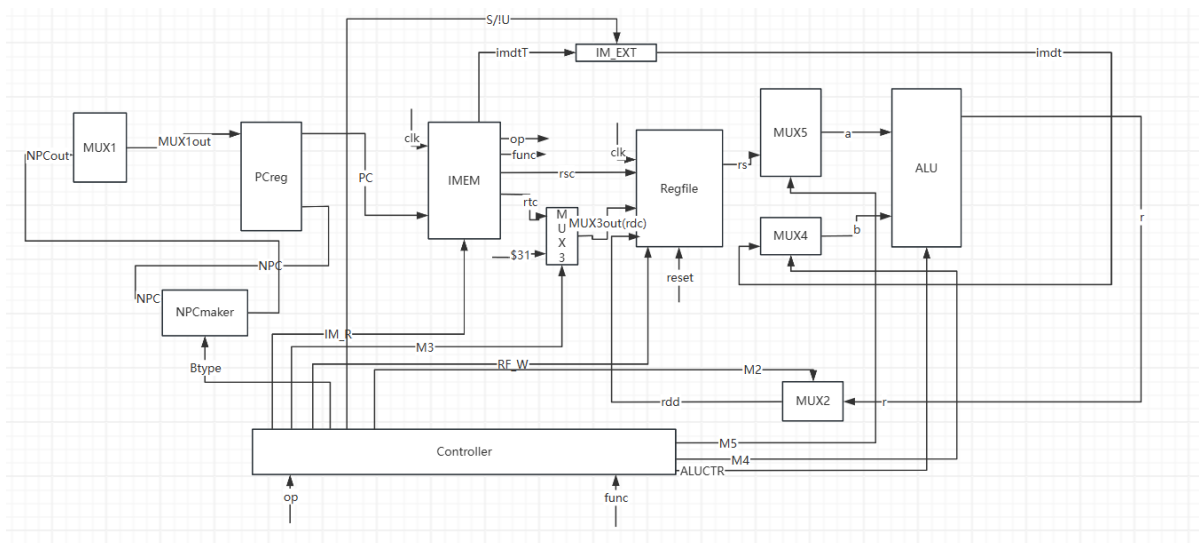


图 2.4 addi 数据通路

在运行 addi 的过程中，会执行如下动作：在时钟的上升沿：

- PC->IMEM
- op->controller
- func->controller
- rsc->regfile
- rtc->regfile
- rs->alu
- imdtT->imdt_EXT
- imdt->alu
- alu 根据输入信号决定执行的指令
- r->rt 更新 rt 的数据

在时钟的下降沿 npc-> PCreg

2.1.5 lw 数据通路

lw 拥有独特的数据通路

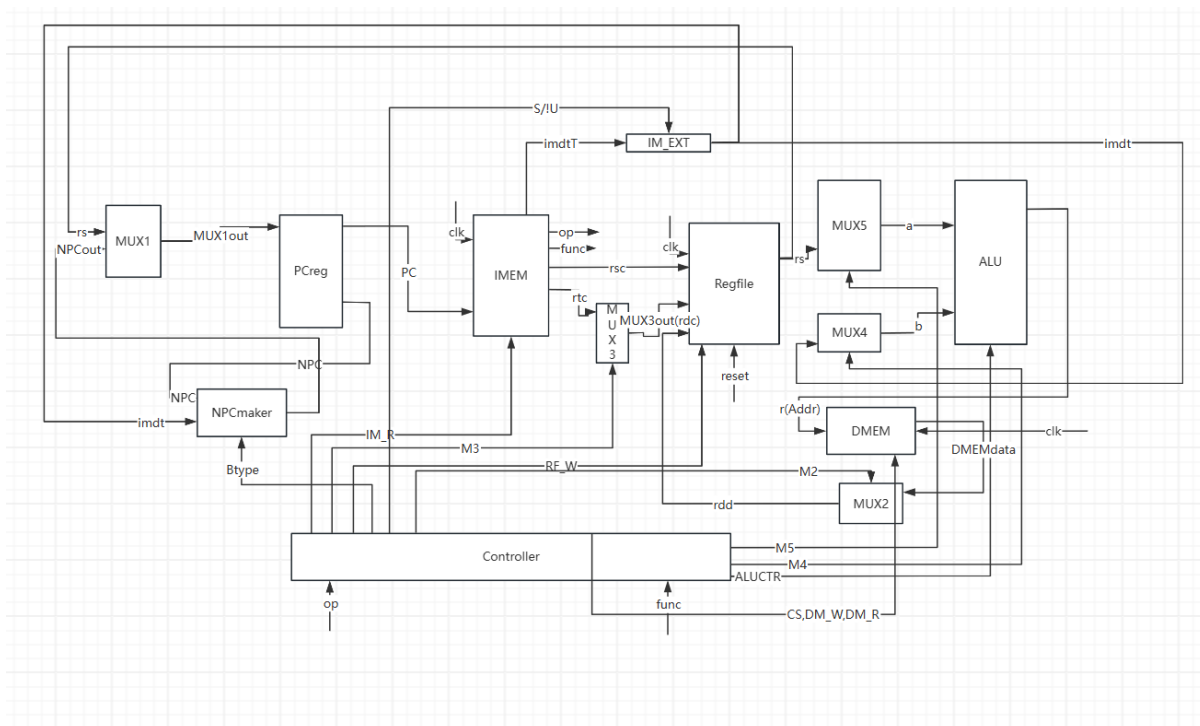


图 2.5 lw 数据通路

在运行 lw 的过程中，会执行如下动作：在时钟的上升沿：

- PC->IMEM
- op->controller
- func->controller
- rsc->regfile
- rtc->regfile
- rs->alu
- imdtT->imdt_EXT
- imdt->rs
- alu 执行有符号加法
- alu->DMEMaddr
- DMEMdata->rt

在时钟的下降沿 npc-> PCreg

2.1.6 sw 数据通路

sw 拥有独特的数据通路

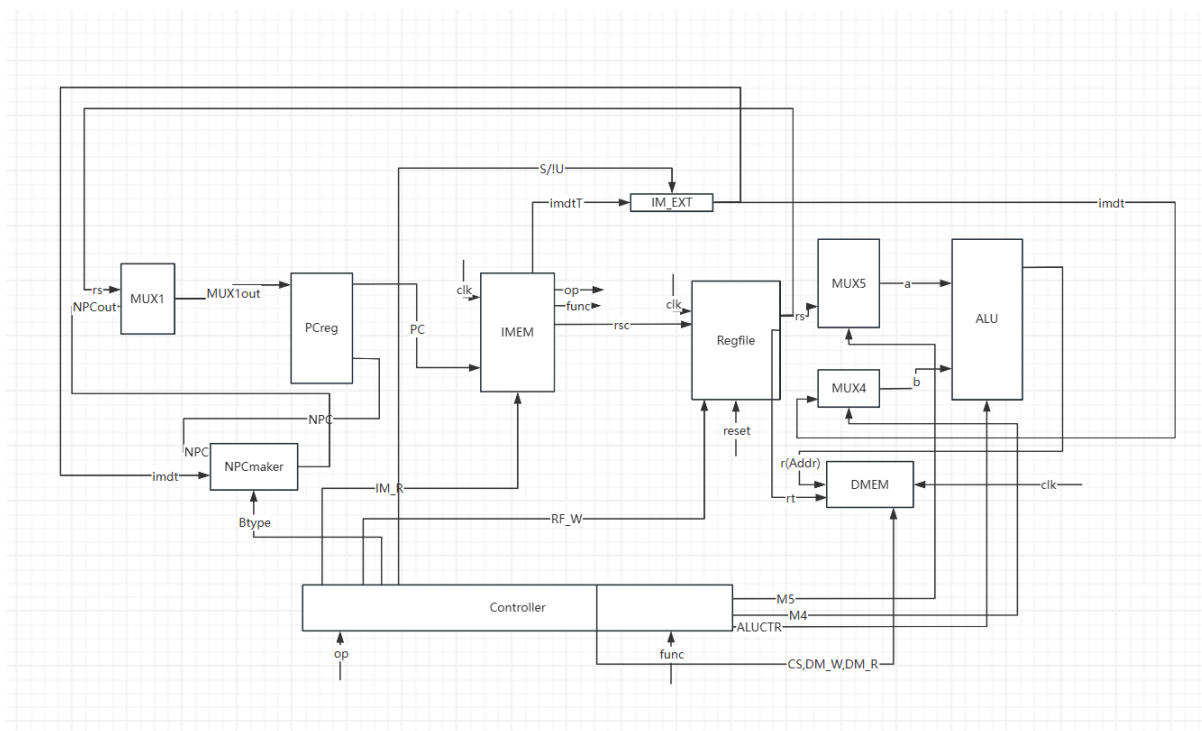


图 2.6 sw 数据通路

在运行 sw 的过程中，会执行如下动作：在时钟的上升沿：

- PC->IMEM
- op->controller
- func->controller
- rsc->regfile
- rtc->regfile
- rs->alu
- imdtT->imdt_EXT
- imdt->rs
- alu 执行有符号加法
- alu->DMEMaddr
- rt->DMEMdata

在时钟的下降沿 npc-> PCreg

2.1.7 beq 数据通路

beq 拥有独特的数据通路

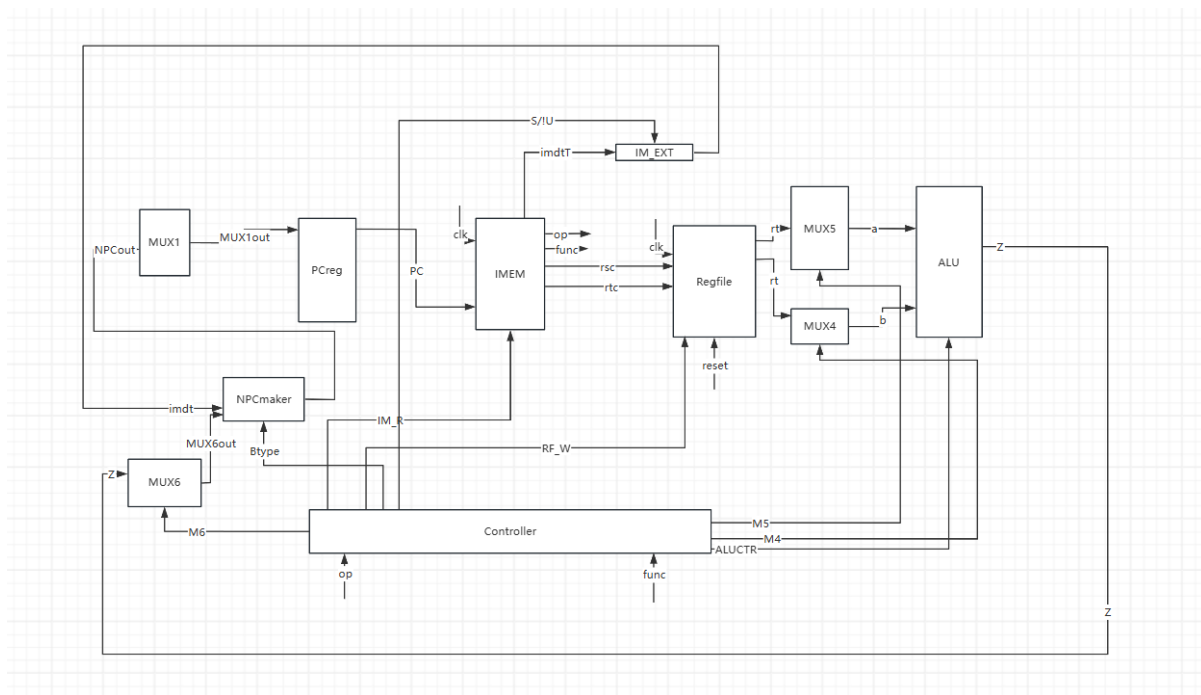


图 2.7 beq 数据通路

在运行 beq 的过程中，会执行如下动作：在时钟的上升沿：

- PC->IMEM
- op->controller
- func->controller
- rsc->regfile
- rtc->regfile
- rs->alu
- imdtT->imdt_EXT
- imdt->rs
- alu 执行无符号减法
- z->npcmaker
- 根据 z 信号，npc 与 imdt 作有符号加法

在时钟的下降沿 NPCOut-> PCreg

bne 拥有独特的数据通路

bne 拥有独特的数据通路

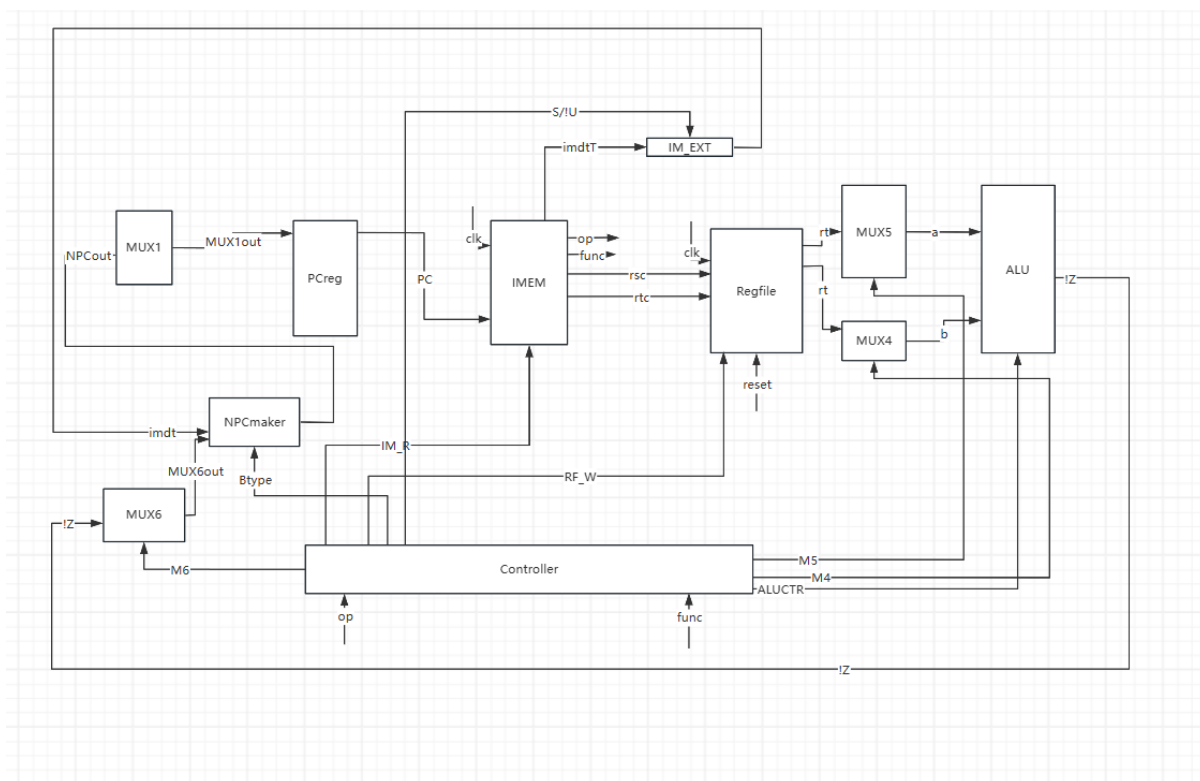


图 2.8 bne 数据通路

在运行 `bne` 的过程中，会执行如下动作：在时钟的上升沿：

- PC->IMEM
- op->controller
- func->controller
- rsc->regfile
- rtc->regfile
- rs->alu
- imdtT->imdt_EXT
- imdt->rs
- alu 执行无符号减法
- !z->npcmaker
- 根据!z 信号, npc 与 imdt 作有符号加法

在时钟的下降沿 NPCout-> PCreg

2.1.9 lui 数据通路

lui 拥有独特的数据通路

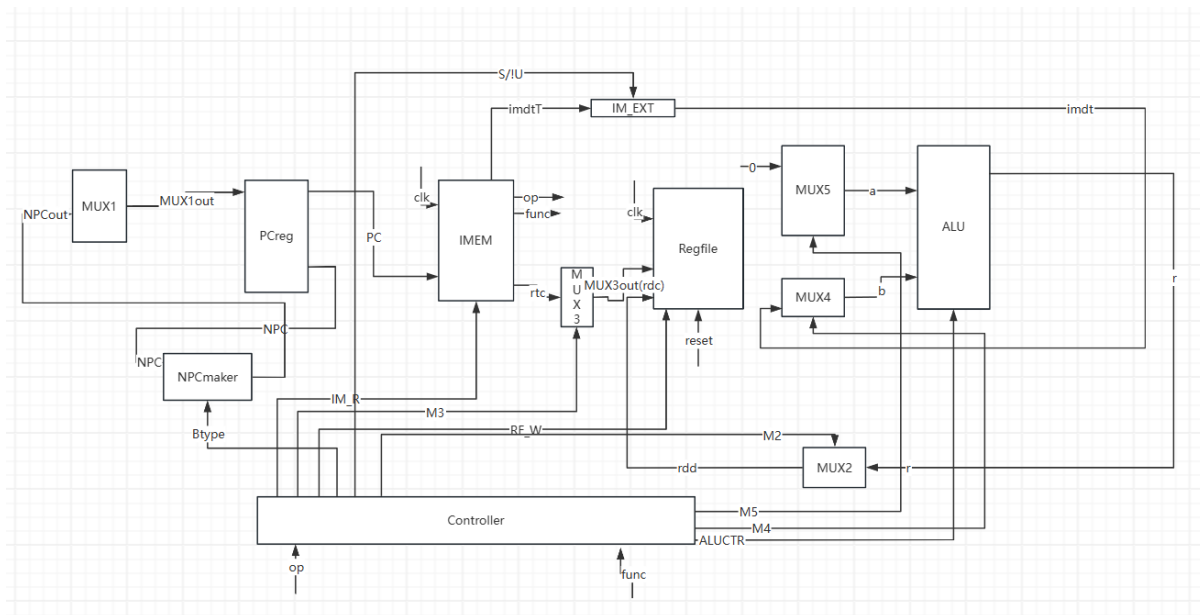


图 2.9 lui 数据通路

在运行 lui 的过程中，会执行如下动作：在时钟的上升沿：

- PC->IMEM
- op->controller
- func->controller
- rdc->regfile
- imdtT->imdtEXT
- imdt->ALU
- \$0->ALU(a)
- ALU 执行 lui r->regfile(rdd)

在时钟的下降沿 NPC-> PCreg

2.1.10 j 数据通路

j 拥有独特的数据通路

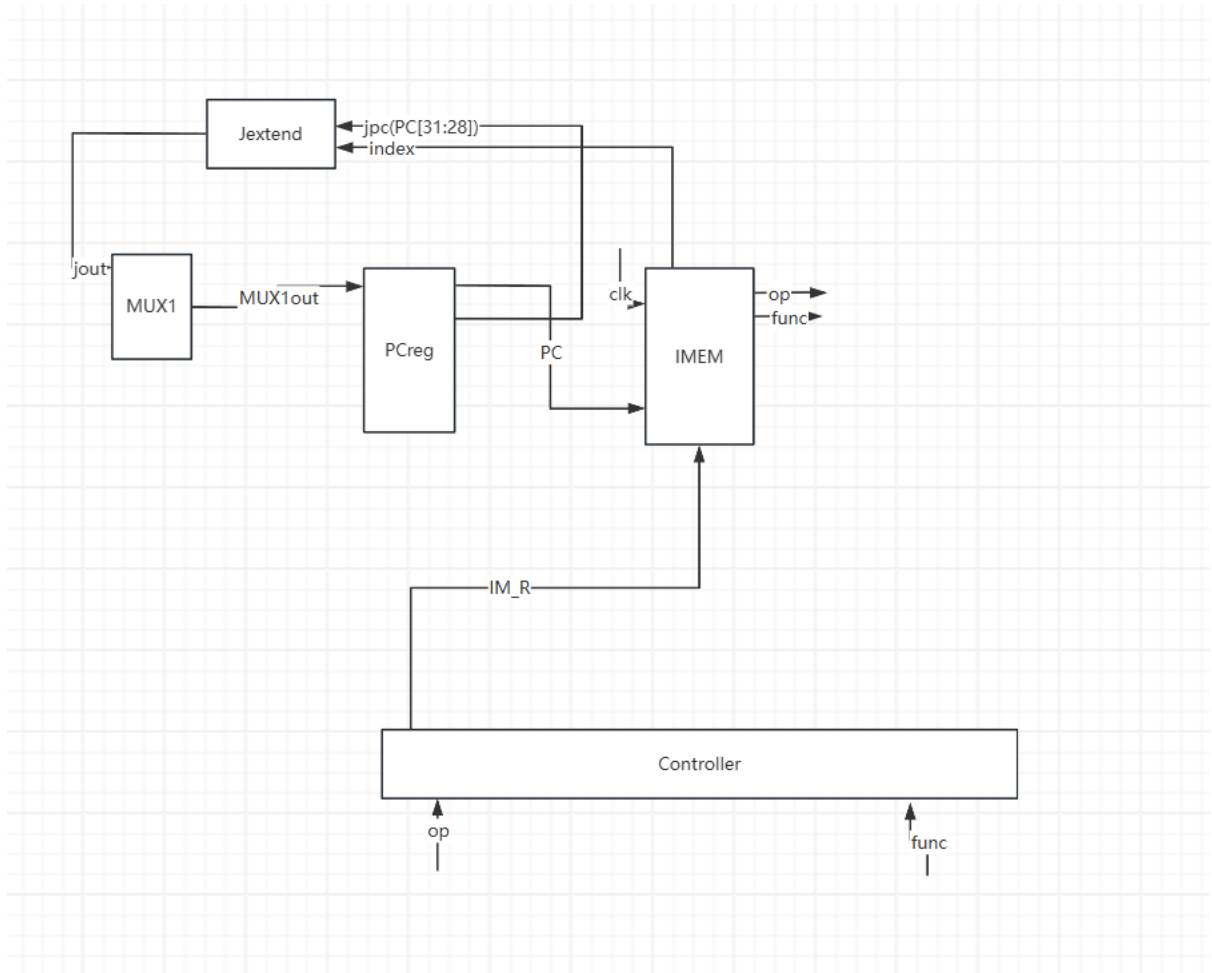


图 2.10 j 数据通路

在运行 j 的过程中，会执行如下动作：在时钟的上升沿：

- PC->IMEM
- op->controller
- func->controller
- index->Jextend

在时钟的下降沿 jout-> PCreg

2.1.11 jal 数据通路

j 拥有独特的数据通路

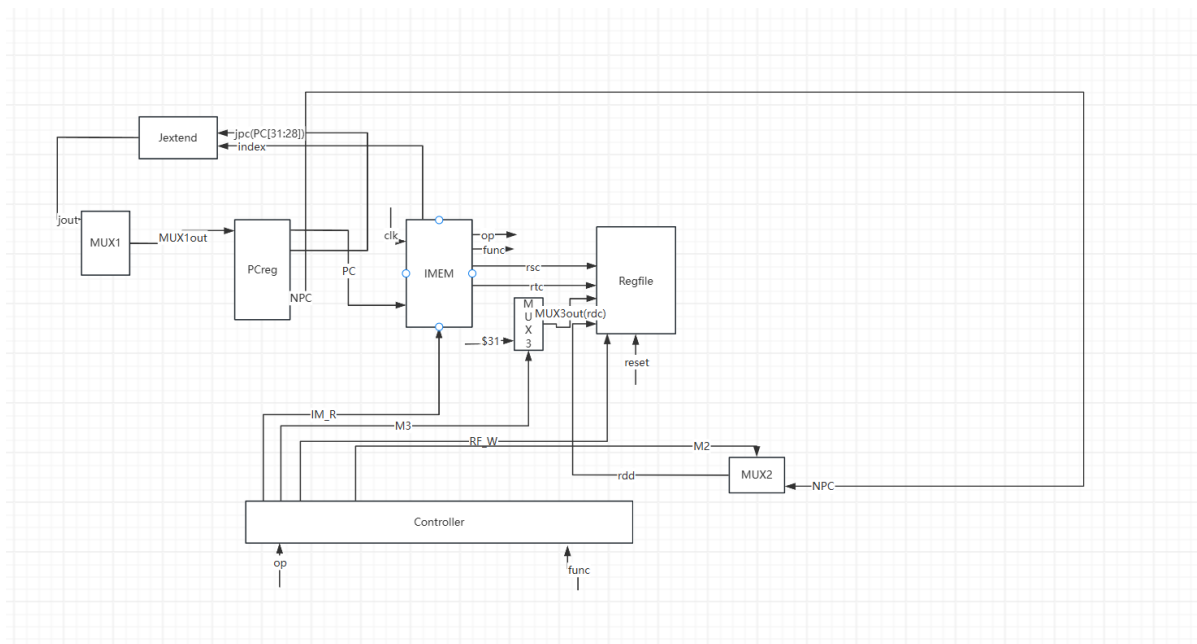


图 2.11 jal 数据通路

在运行 jal 的过程中，会执行如下动作：在时钟的上升沿：

- PC->IMEM
- op->controller
- func->controller
- index->Jextend
- \$31 ->regfile(rdc)
- NPC->regfile(rdd)

在时钟的下降沿 jout-> PCreg

2.2 整体数据通路

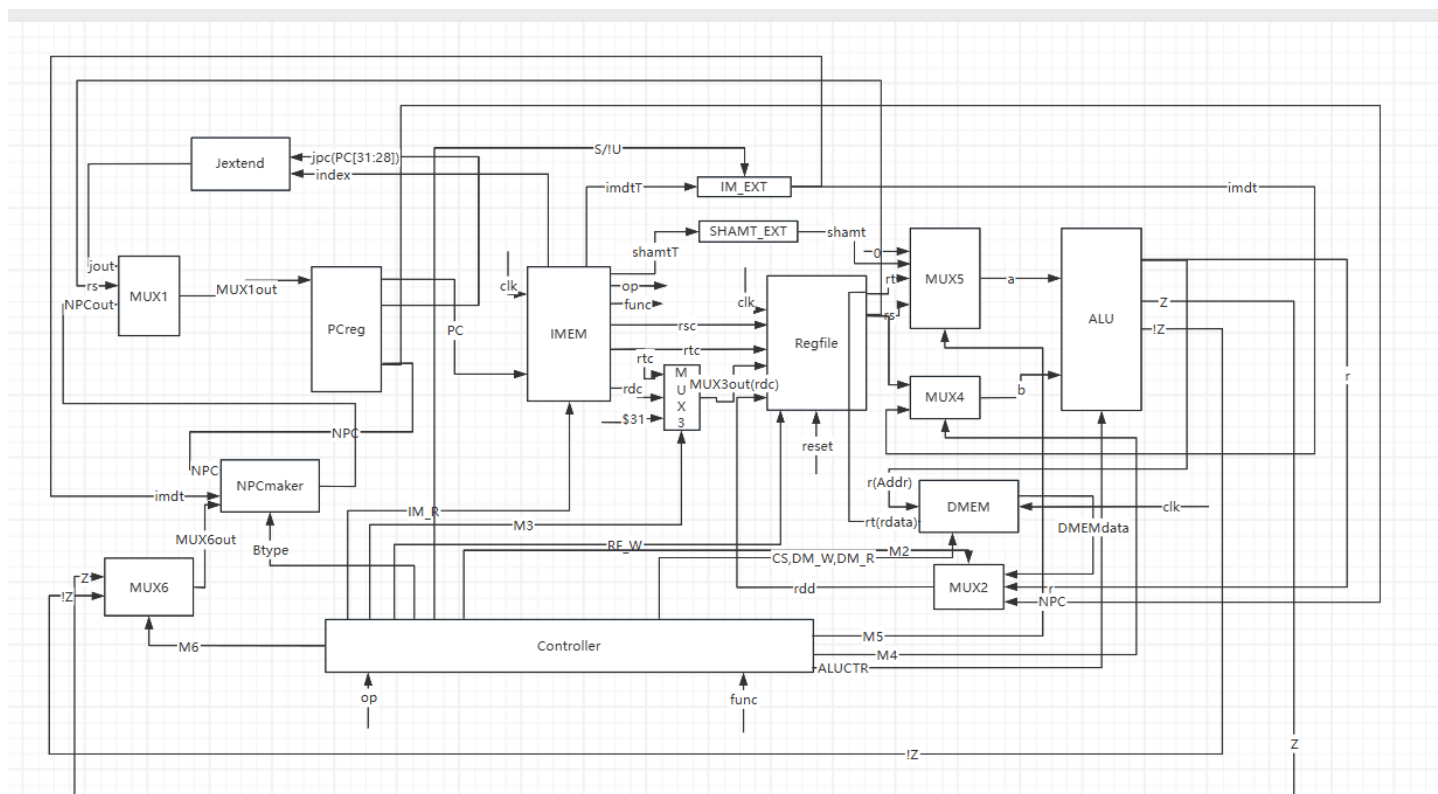


图 2.12 整体数据通路

2.3 每条指令的要求

表 2.1 每条指令的要求

助记符		指令格式		示例		示例含义		操作及其解释	
Bit #	31..26	25..21	20..16	15..11	10..6	5..0			
R-type	op	rs	rt	rd	shamt	func			
add	000000	rs	rt	rd	0	100000	add 1,2,\$3	rd <- rs + rt ; 其中 rs = 2rt = 3, rd=\$1	
addu	000000	rs	rt	rd	0	100001	addu 1,2,\$3	rd <- rs + rt ; 其中 rs = 2rt = 3, rd=\$1, 无符号数	
sub	000000	rs	rt	rd	0	100010	sub 1,2,\$3	rd <- rs - rt ; 其中 rs = 2rt = 3, rd=\$1	
subu	000000	rs	rt	rd	0	100011	subu 1,2,\$3	rd <- rs - rt ; 其中 rs = 2rt = 3, rd=\$1, 无符号数	
and	000000	rs	rt	rd	0	100100	and 1,2,\$3	rd <- rs & rt ; 其中 rs = 2rt = 3, rd=\$1	
or	000000	rs	rt	rd	0	100101	or 1,2,\$3	rd <- rs rt ; 其中 rs = 2rt = 3, rd=\$1	
xor	000000	rs	rt	rd	0	100110	xor 1,2,\$3	rd <- rs xor rt ; 其中 rs = 2rt = 3, rd=\$1(异或)	
nor	000000	rs	rt	rd	0	100111	nor 1,2,\$3	rd <- not(rs rt) ; 其中 rs = 2rt = 3, rd=\$1(或非)	
sll	000000	rs	rt	rd	0	101010	sll 1,2,\$3	if (rs < rt) rd=1 else rd=0 ; 其中 rs = 2rt = 3, rd=\$1	
sll	000000	rs	rt	rd	0	101011	sll 1,2,\$3	if (rs < rt) rd=1 else rd=0 ; 其中 rs = 2rt = 3, rd=\$1 (无符号数)	
srl	000000	0	rt	rd	shamt	000000	srl 1,2,10	if (rs < rt) rd=1 else rd=0 ; 其中 rs = 2rt = 3, rd=\$1 (无符号数)	
srl	000000	0	rt	rd	shamt	000010	srl 1,2,10	rd <- rt >>shamt ; (logical), 其中 rt=2, rd=1	
sra	000000	0	rt	rd	shamt	000011	sra 1,2,10	rd <- rt >>shamt ; (arithmetic) 注意符号位保留, 其中 rt=2, rd=1	
slv	000000	rs	rt	rd	0	000100	slv 1,2,\$3	rd <- rt <<rs ; 其中 rs = 3rt = 2, rd=\$1	
srav	000000	rs	rt	rd	0	000110	srav 1,2,\$3	rd <- rt >>rs ; (logical) 其中 rs = 3rt = 2, rd=\$1	
srav	000000	rs	rt	rd	0	000111	srav 1,2,\$3	rd <- rt >>rs ; (arithmetic) 注意符号位保留其中 rs = 3rt = 2, rd=\$1	
jr	000000	rs	0	0	0	001000	jr \$31	PC <- rs	
J-type	op	rs	rt	immediate					
addi	001000	rs	rt	immediate			addi 1,2,100	rt <- rs + (sign-extend)immediate ; 其中 rt=1, rs = 2	
addiu	001001	rs	rt	immediate			addiu 1,2,100	rt <- rs + (zero-extend)immediate ; 其中 rt=1, rs = 2	
andi	001100	rs	rt	immediate			andi 1,2,10	rt <- rs & (zero-extend)immediate ; 其中 rt=1, rs = 2	
ori	001101	rs	rt	immediate			andi 1,2,10	rt <- rs (zero-extend)immediate ; 其中 rt=1, rs = 2	
xori	001110	rs	rt	immediate			andi 1,2,10	rt <- rs xor (zero-extend)immediate ; 其中 rt=1, rs = 2	
lui	001111	0	rt	immediate			lui 1,100	rt <- rs xor (zero-extend)immediate ; 其中 rt=1, rs = 2	
lw	100011	rs	rt	immediate			lw 1,10(2)	rt <- memory[rs + (sign-extend)immediate] ; rt=1, rs = 2	
sw	101011	rs	rt	immediate			sw 1,10(2)	memory[rs + (sign-extend)immediate] <- rt ; rt=1, rs = 2	
beq	000100	rs	rt	immediate			beq 1,2,10	if (rs == rt) PC <- PC+4 + (sign-extend)immediate<<2	
bne	000101	rs	rt	immediate			bne 1,2,10	if (rs != rt) PC <- PC+4 + (sign-extend)immediate<<2	
slti	001010	rs	rt	immediate			slli 1,2,10	if (rs < (sign-extend)immediate) rt=1 else rt=0 ; 其中 rs = 2rt = 1	
sltiu	001011	rs	rt	immediate			sliu 1,2,10	if (rs < (zero-extend)immediate) rt=1 else rt=0 ; 其中 rs = 2rt = 1	
J-type	op	address							
j	000010	address					j 10000	PC <- (PC+4)[31..28],address,0,0 ; address=10000/4	
jal	000011	address					jal 10000	\$31<-PC+4; PC <- (PC+4)[31..28],address,0,0 ; address=10000/4	

3 CPU 控制部件设计

3.1 操作时间表

|
|
|
|
|
|
|
|
|
|
装
|
|
|
|
|
订
|
|
|
|
|
线
|
|
|
|
|
|
|
|
|
|
|

装 订 线

cpu 指令集	op	func	PC_CLK	M1	IM_R	RF_W	RF_CLK	S/U	M2	M4	CS	DM_W	DM_R	CLK	ALUCTR	Btype	M5	M3	M6
add	000000	100000	1	00	1	1	1	0	00	0	0	0	0	1	0010	0	00	00	0
addu	000000	100001	1	00	1	1	1	0	00	0	0	0	0	1	0000	0	00	00	0
sub	000000	100010	1	00	1	1	1	0	00	0	0	0	0	1	0011	0	00	00	0
subu	000000	100011	1	00	1	1	1	0	00	0	0	0	0	1	0001	0	00	00	0
and	000000	100100	1	00	1	1	1	0	00	0	0	0	0	1	0100	0	00	00	0
or	000000	100101	1	00	1	1	1	0	00	0	0	0	0	1	0101	0	00	00	0
xor	000000	100110	1	00	1	1	1	0	00	0	0	0	0	1	0110	0	00	00	0
nor	000000	100111	1	00	1	1	1	0	00	0	0	0	0	1	0111	0	00	00	0
slt	000000	101010	1	00	1	1	1	0	00	0	0	0	0	1	1011	0	00	00	0
sltu	000000	101011	1	00	1	1	1	0	00	0	0	0	0	1	1010	0	00	00	0
sll	000000	000000	1	00	1	1	1	0	00	0	0	0	0	1	1110	0	01	00	0
srl	000000	000010	1	00	1	1	1	0	00	0	0	0	0	1	1101	0	01	00	0
sra	000000	000011	1	00	1	1	1	0	00	0	0	0	0	1	1100	0	01	00	0
slv	000000	000100	1	00	1	1	1	0	00	0	0	0	0	1	1110	0	00	00	0
srlv	000000	000110	1	00	1	1	1	0	00	0	0	0	0	1	1101	0	00	00	0
srav	000000	000111	1	00	1	1	1	0	00	0	0	0	0	1	1100	0	00	00	0
jr	000000	001000	1	10	1	0	1	0	00	0	0	0	0	1	0000	0	00	00	0
addi	001000	000000	1	00	1	1	1	1	00	1	0	0	0	1	0010	0	00	10	0
addiu	001001	000000	1	00	1	1	1	1	00	1	0	0	0	1	0000	0	00	10	0
andi	001100	000000	1	00	1	1	1	0	00	1	0	0	0	1	0100	0	00	10	0
ori	001101	000000	1	00	1	1	1	0	00	1	0	0	0	1	0101	0	00	10	0
xori	001110	000000	1	00	1	1	1	0	00	1	0	0	0	1	0110	0	00	10	0
lw	100011	000000	1	00	1	1	1	1	01	1	1	0	1	1	0010	0	00	10	0
sw	101011	000000	1	00	1	0	1	1	00	1	1	1	0	1	0010	0	00	10	0
beq	000100	000000	1	00	1	0	1	1	00	1	0	0	0	1	0011	1	11	00	1
bne	000101	000000	1	00	1	0	1	1	00	1	0	0	0	1	0011	1	11	00	0
slti	001010	000000	1	00	1	1	1	1	00	0	0	0	0	1	1011	0	00	10	0
sltiu	001011	000000	1	00	1	1	1	0	00	0	0	0	0	1	1010	0	00	10	0
lui	001111	000000	1	00	1	1	1	0	00	1	0	0	0	1	1000	0	10	10	0
j	000010	000000	1	01	1	0	1	0	00	0	0	0	0	1	0000	0	00	00	0
jal	000011	000000	1	01	1	1	1	0	10	0	0	0	0	1	0000	0	00	01	0

3.2 各个指令的逻辑表达式

Listing 1 **logic.v**

```

1  assign M1=(jrT)?2'b10:(jt||jalT)?2'b01:2'b00;
2  assign IM_R=1;
3  assign RF_W=(jrT|swT|beqT|bneT|jt)?0:1;
4  assign su=(addiT|addiuT|lwT|swT|beqT|bneT|sltiT)?1:0;
5  assign M2=(lwT)?2'b01:(jalT)?2'b10:2'b00;
6  assign M4=(addiT|addiuT|andiT|oriT|xoriT|lwT|swT|sltiT|sltiuT|luiT)?1:0;
7  assign CS=(lwT|swT)?1:0;
8  assign DM_W=(swT)?1:0;
9  assign DM_R=(lwT)?1:0;
10 assign Btype=(beqT|bneT)?1:0;
11 assign aluc=
12 (addT|addiT)?add_aluc:
13 (adduT|addiuT)?addu_aluc:
14 (subT|subuT|bneT|beqT)?sub_aluc:
15 (andT|andiT)?and_aluc:
16 (orT|oriT)?or_aluc:
17 (xorT|xoriT)?xor_aluc:
18 (norT)?nor_aluc:
19 (sltT|sltiT)?slt_aluc:
20 (sltuT|sltiuT)?sltu_aluc:
21 (sllT|sllvT)?sll_aluc:
22 (srlT|srlvT)?srl_aluc:
23 (sraT|sravT)?sra_aluc:
24 (luiT)?lui_aluc:0;
25 assign M5=(sllT|srlT|sraT)?2'b01:(luiT)?2'b10:2'b00;
26 assign M6=(beqT)?1:0;
27 assign M3=(lwT|swT|sltiT|sltiuT|luiT|addiT|addiuT|andiT|oriT|xoriT)?2'b10:(jalT)?2'b01:2'b00;

```

4 CPU 前仿真测试结果

4.1 modelsim 仿真

在 modelsim 仿真当中，由于没有注意到实验报告当中对于贴图的要求，只有部分的图片得以保留，截图的最初目的是记录自己完成了这个 cpu 而非为了实验报告，因此本节相对缺失。

我们知道，在一个 module 当中，有任意多个 output wire 不被连接都是可以接受的，因此我为了方便寻找自己 cpu 模块设计当中的问题，将所有的控制线，数据传输线，地址线，全部添加到了顶层模块当中进行了输出，这样大大降低了 debug 的成本。由于我的代码没有借鉴任何学长的代码，单纯由自己完成，因此细节上的错误很多，不采用这样的办法去 debug 是没有可能完成整个流程的。

在顶层模块当中，我们发现 mars 当中展示的是本次执行的指令，但是给出的 _246tb_ex9_tb 当中，输出的是下一个周期要执行的指令，为了弥合这个误差。我设置了 PCreg 在下降沿更新，DMEM 以及 REGfile 在上升沿更新，同时设置 testbench 当中的输出模块在时钟为 0 的时候进行输出。

在 debug 的过程中，我曾经遇到以下问题：

1 很令人烦恼的一方面，老师给出的操作表并不合理。我们并不知道寄存器与代码段中的 rs, rt, rd 的对应关系，这带来了控制部件设计的混乱。因此我特意在第二节的表 2.1 最后加入了这一条。

2 数据通路到底该怎么话，操作时间表到底怎么用，我觉得可以参考《数字逻辑与组成原理实践教程》249-268 页的内容完成设计。建议手绘一份流程图设计，再进行下一步的操作。如下图所示

3 要考虑地址的地方有很多，包括 jr, j, jal, beq, bne。那么到底如何存储地址呢？在 pc 当中，我直接采用了哈佛架构，如实验文档中所说的那样设计。在 j 指令和 jr 指令的时候，我将输入的地址减去了 32'b00400000 才传入 pcreg 当中。因此，在 jal 存储地址的时候，就要把失去的再加回来。在 beq 和 bne 指令当中，由于是相对寻址，因此不需要如此设计。

4 很令人厌烦的一点，addiu 采用的是有符号扩展，我请问这个设计真的是设计者期望的吗？

5 除了 shamt 以外的所有立即数都是通过 alu 的 b 口送入，但是只有 shamt 是通过 alu 的 a 口送入，这个在设计中需要尤为重视。

6 数据通路最为特殊的是 lui 指令，因为在 alu 当中是有这条指令的。正常来讲直接把立即数送入寄存器即可，但是我们还要送到 alu 当中做第二次拓展。我直接使用了上个学期的 alu，好吧，我遵从过去的我的意愿。

4.2 整体数据通路

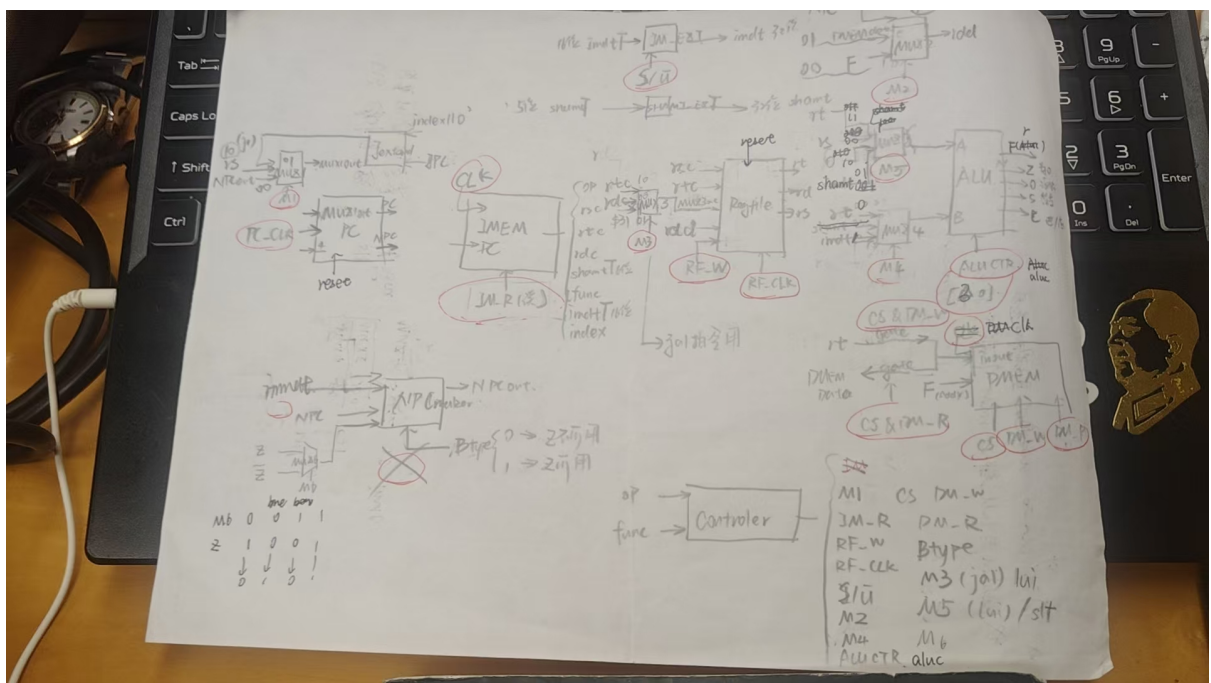


图 4.1 手绘流程图

4.3 仿真流程

打开 mars 生成二进制文本文件以及程序运行结果

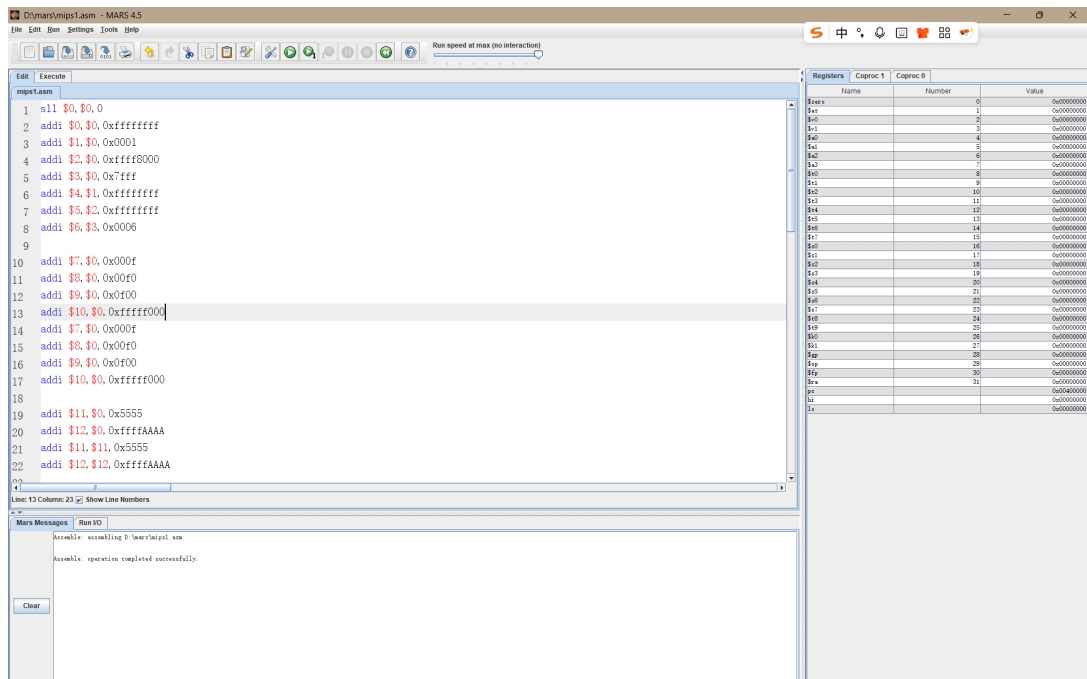


图 4.2 sim

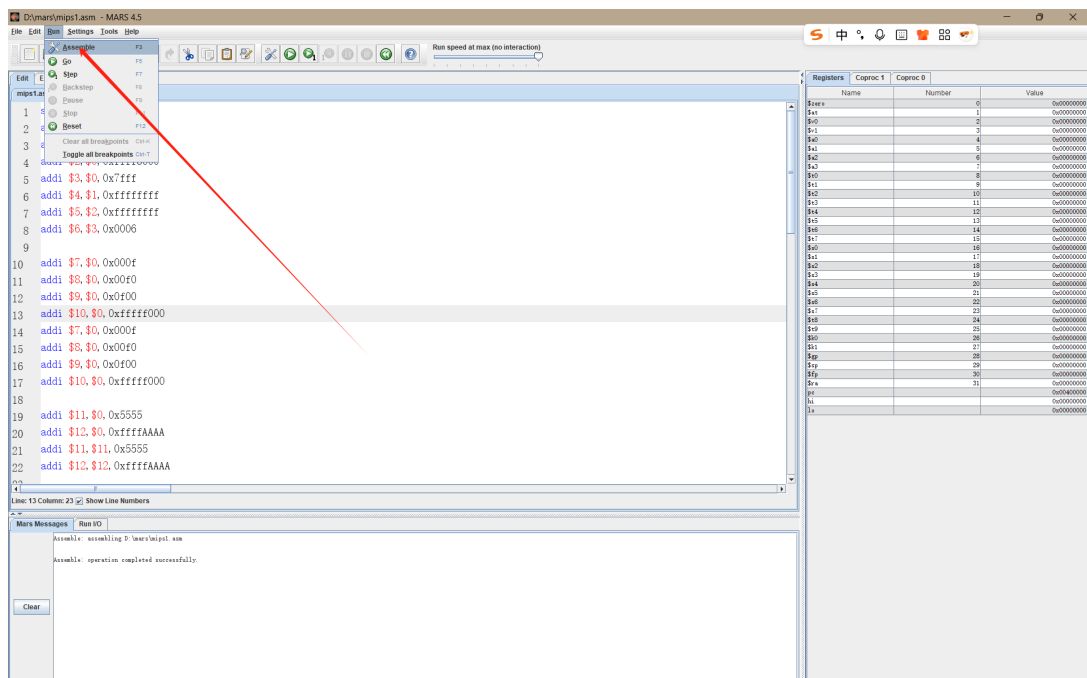


图 4.3 sim

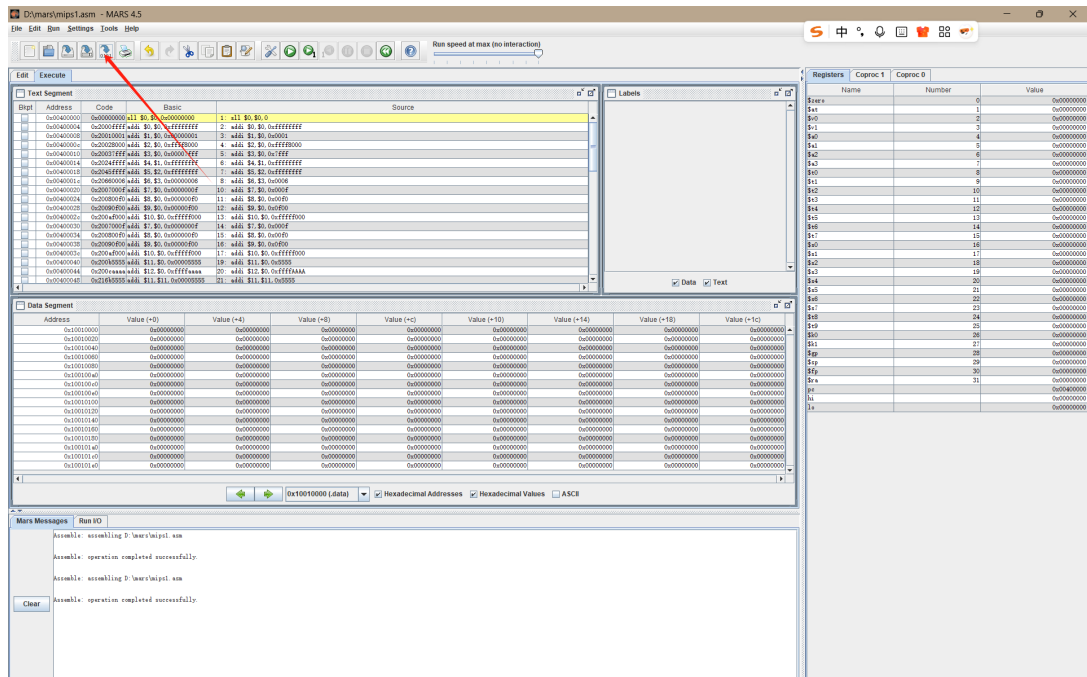


图 4.4 sim

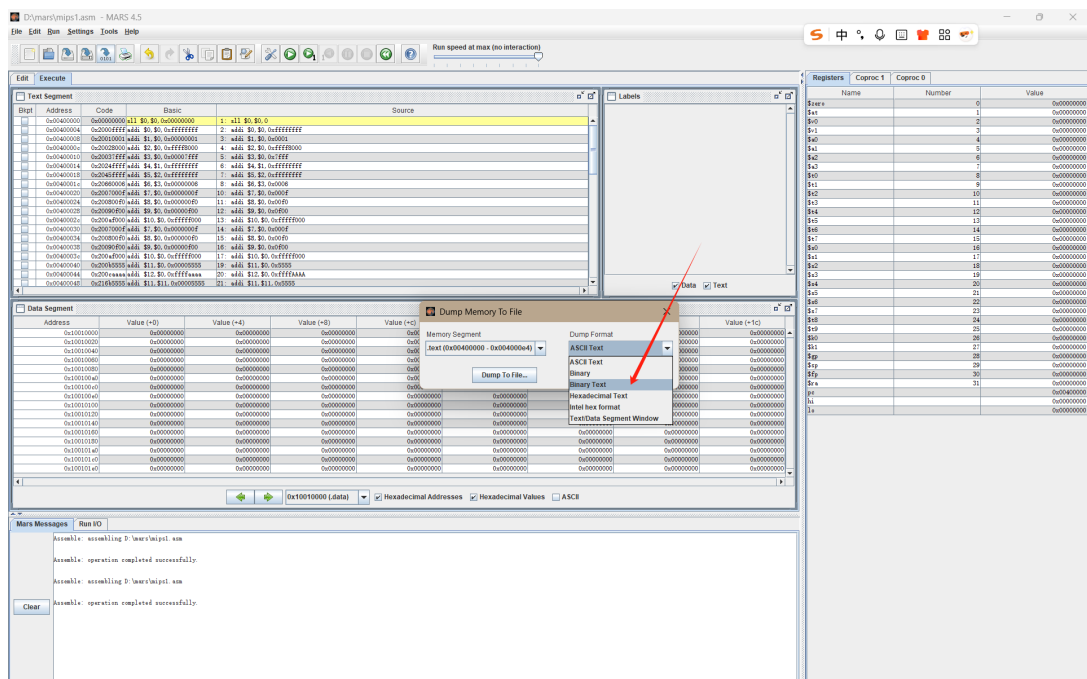


图 4.5 sim

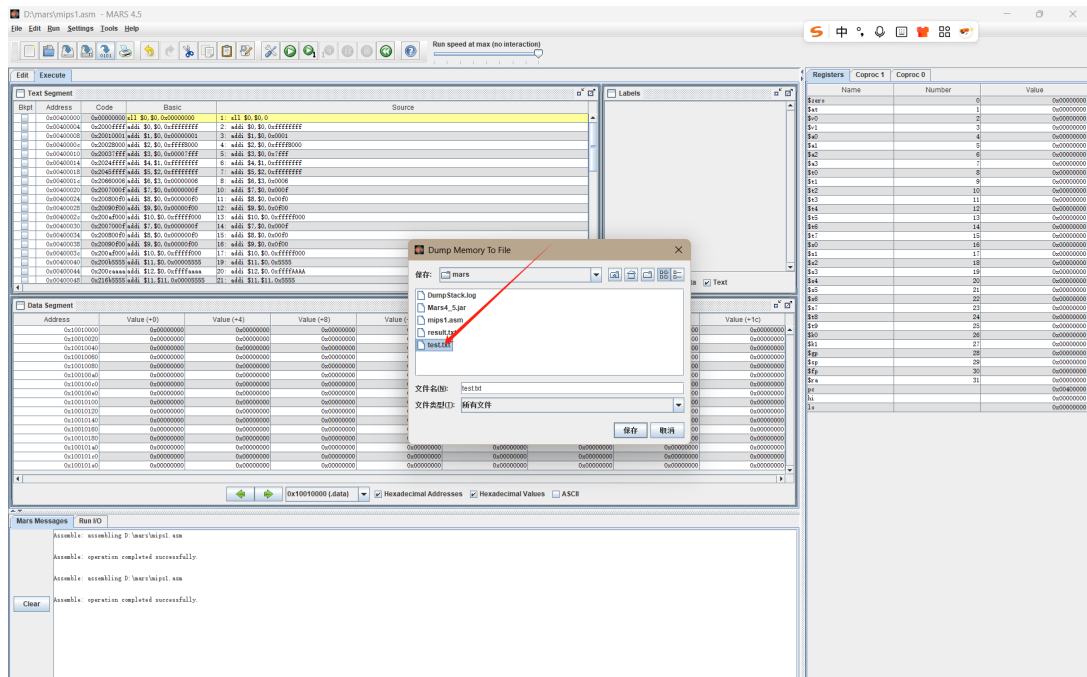


图 4.6 sim

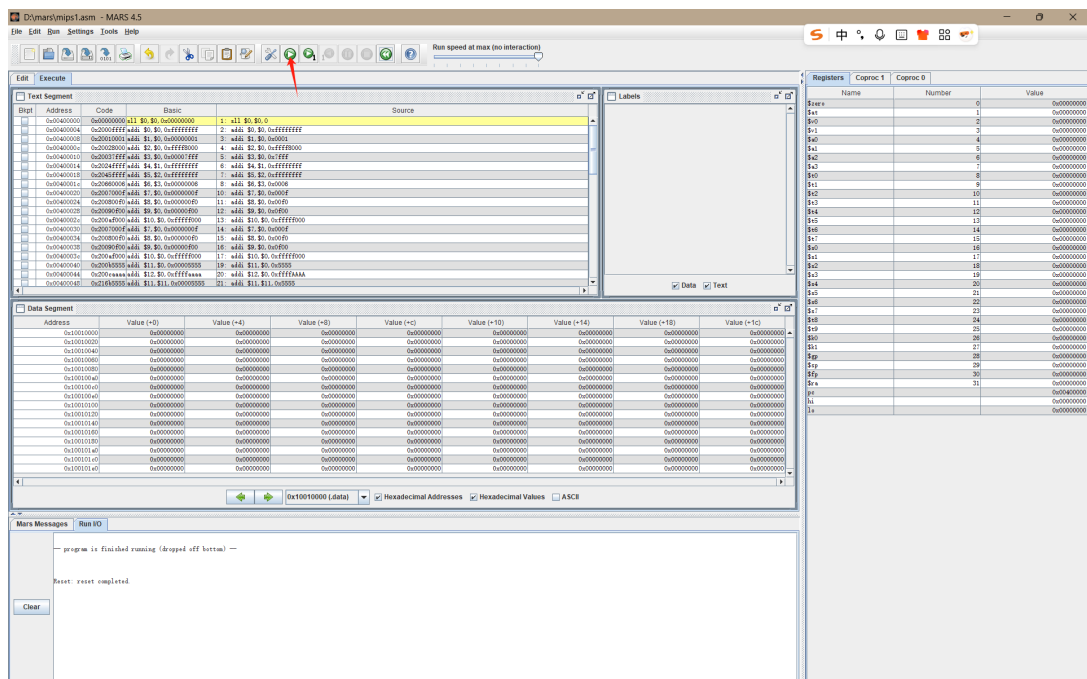


图 4.7 sim

将仿真运行到末尾

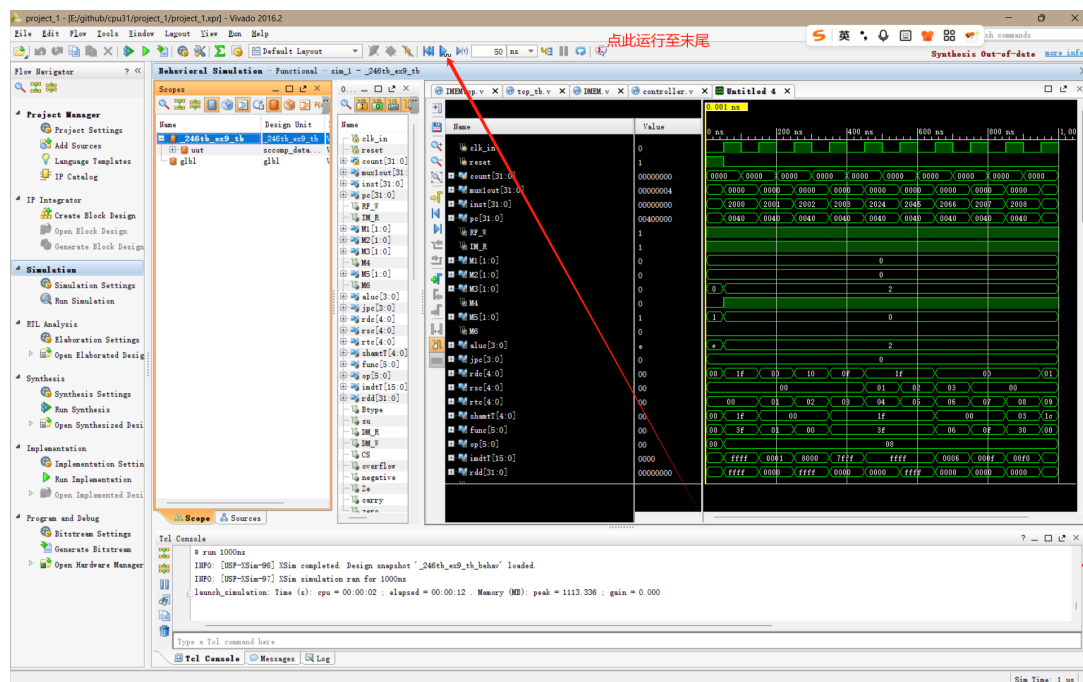


图 4.8 sim

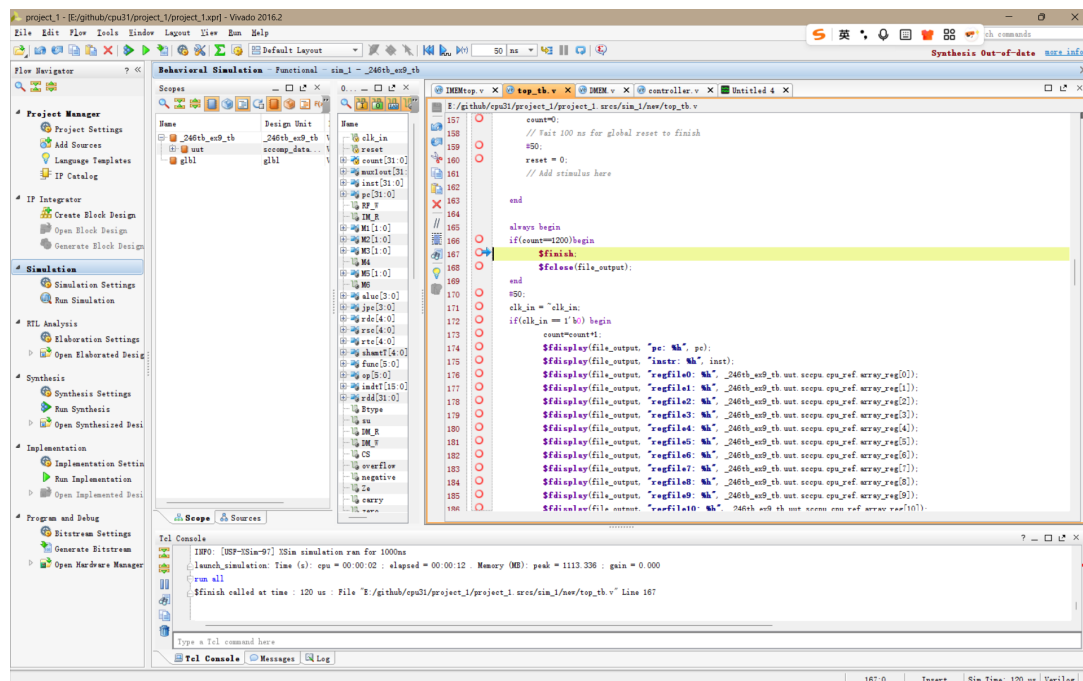


图 4.9 sim

使用上个学期沈坚老师提供的 txt_compare 工具进行文本比对

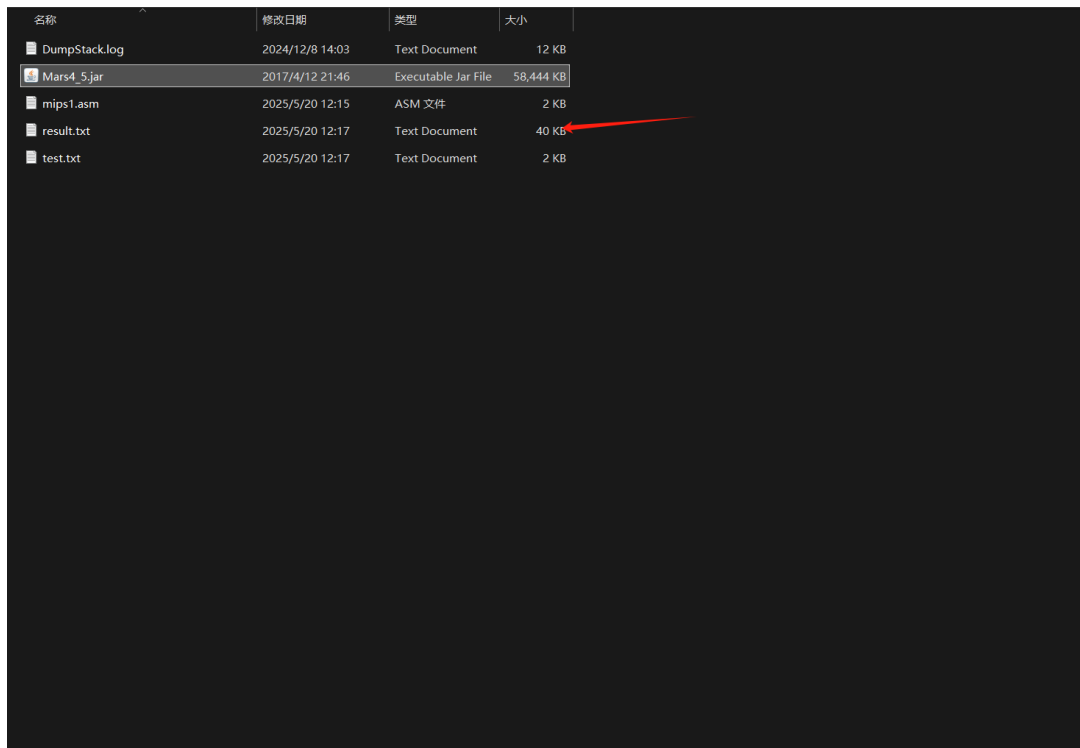


图 4.10 sim

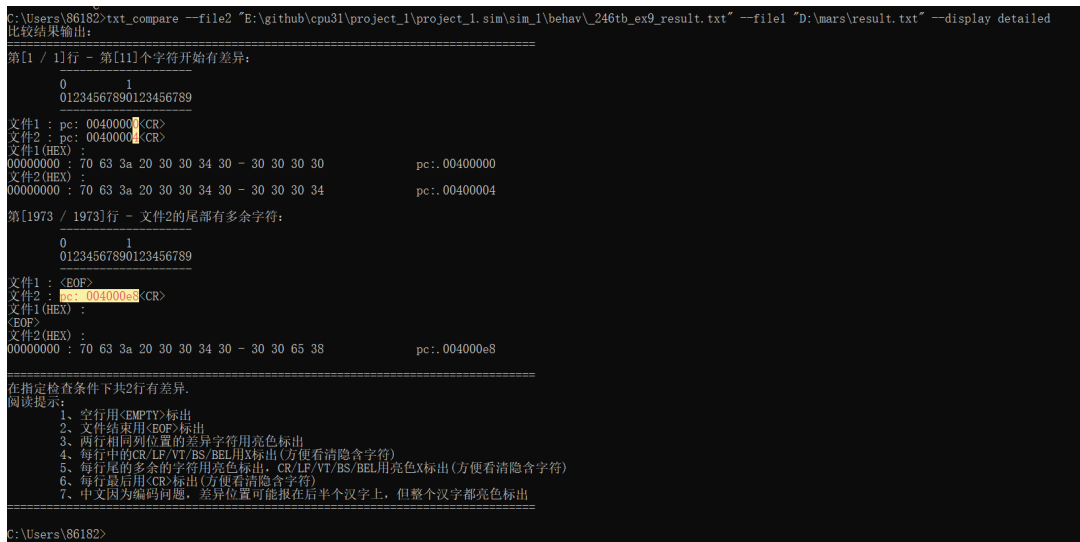


图 4.11 sim

5 CPU 后仿真

5.1 后仿真遇到的问题

后仿真我遇到了一个严重的问题，在产生的波形图当中，自第一个周期起，后面的再也没有办法产生有效的信号。如下图所示。

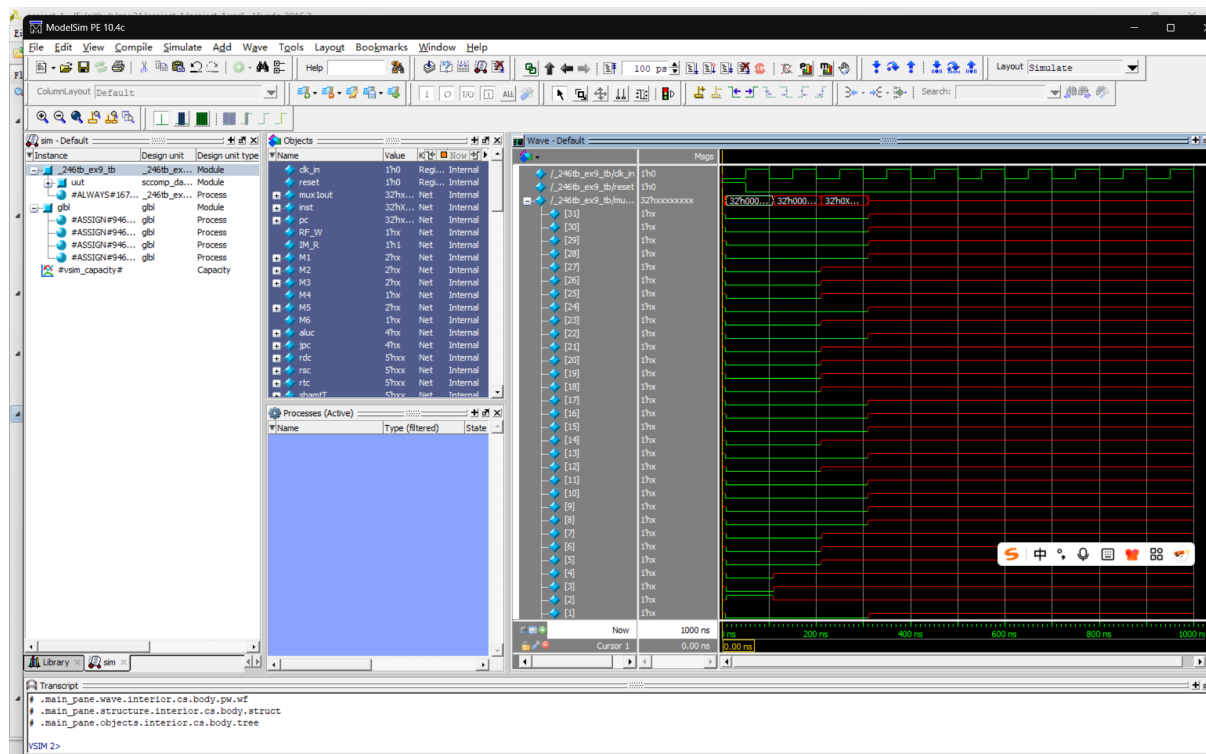


图 5.1 post

那么为什么会产生这个问题呢？我们可以看到，pc 在两个周期后才进入完全的异常。再看 pc 当中的信号，只有 1 才会发生错误，0 没有发生错误。我最初怀疑是后仿真的过程中，由于我的 mux 当中直接使用了线而没有使用 reg 进行高阻抗设置，导致电流反向传播进入 pc 当中而导致 pc 短路。但是实际当我添加了所有的 reg 之后并没有解决这个问题。随后我排查 pc 的时候，想到上个学期期末遇到了相同的问题，也是在 1 的位置才会出错。我甚至考虑了我的输出线和输入线产生了冲突导致电流反向传播产生了短路。经过一系列的检查，我发现，出现问题的是因为我在 pcreg 搭建的过程中，出现了两个驱动同时驱动同一个 pcreg 的现象。如下代码所示：

Listing 2 pcregwrong.v

```

1  `timescale 1ns/1ps
2  module pcreg(
3      input pc_clk,
4      input reset,
5      input [31:0] mux1out,
6      output [31:0] npc,
7      output reg [31:0] pc,
8      output [3:0] jpc
9
10 );
11
12 assign jpc = pc[31:28];
13 always @(posedge reset) begin
14     if (reset) begin
15         pc = 0;
16     end
17 end
18 always @(negedge pc_clk) begin
19     if (!reset) begin
20         pc = mux1out;
21     end
22 end
23
24 assign npc = pc + 32'b4;
25
26 endmodule

```

因此要把他们放到同一个驱动当中，使用阻塞赋值完成这个过程。

Listing 3 pcregcorrect.v

```

1  `timescale 1ns/1ps
2  module pcreg(
3      input pc_clk,
4      input reset,
5      input [31:0] mux1out,
6      output [31:0] npc,
7      output reg [31:0] pc,
8      output [3:0] jpc
9
10 );

```

```

10
11 assign jpc = pc[31:28];
12 always @(posedge pc_clk or posedge reset) begin
13     if (reset) begin
14         pc <= 32'b0;
15     end else begin
16         pc <= mux1out;
17     end
18 end
19
20 assign npc = pc + 32'd4;
21
22 endmodule

```

装

订

线

5.2 后仿真效果图

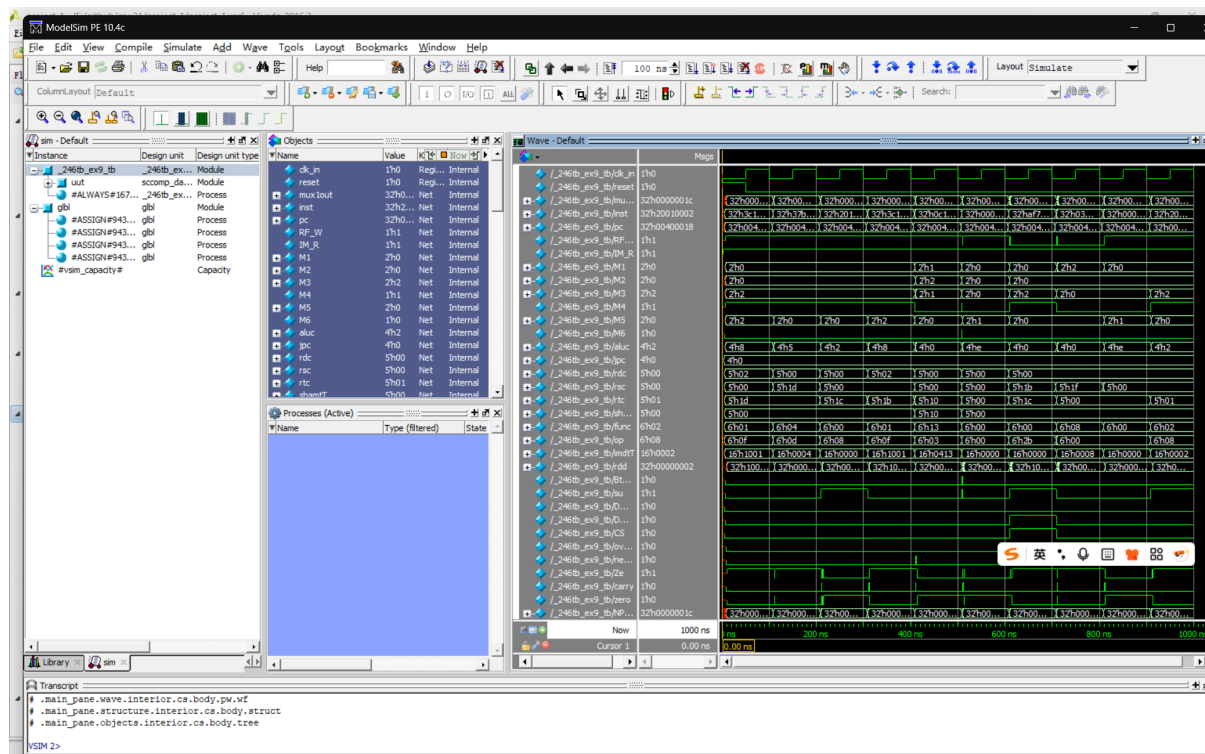


图 5.2 post

5.3 CPU 下板结果

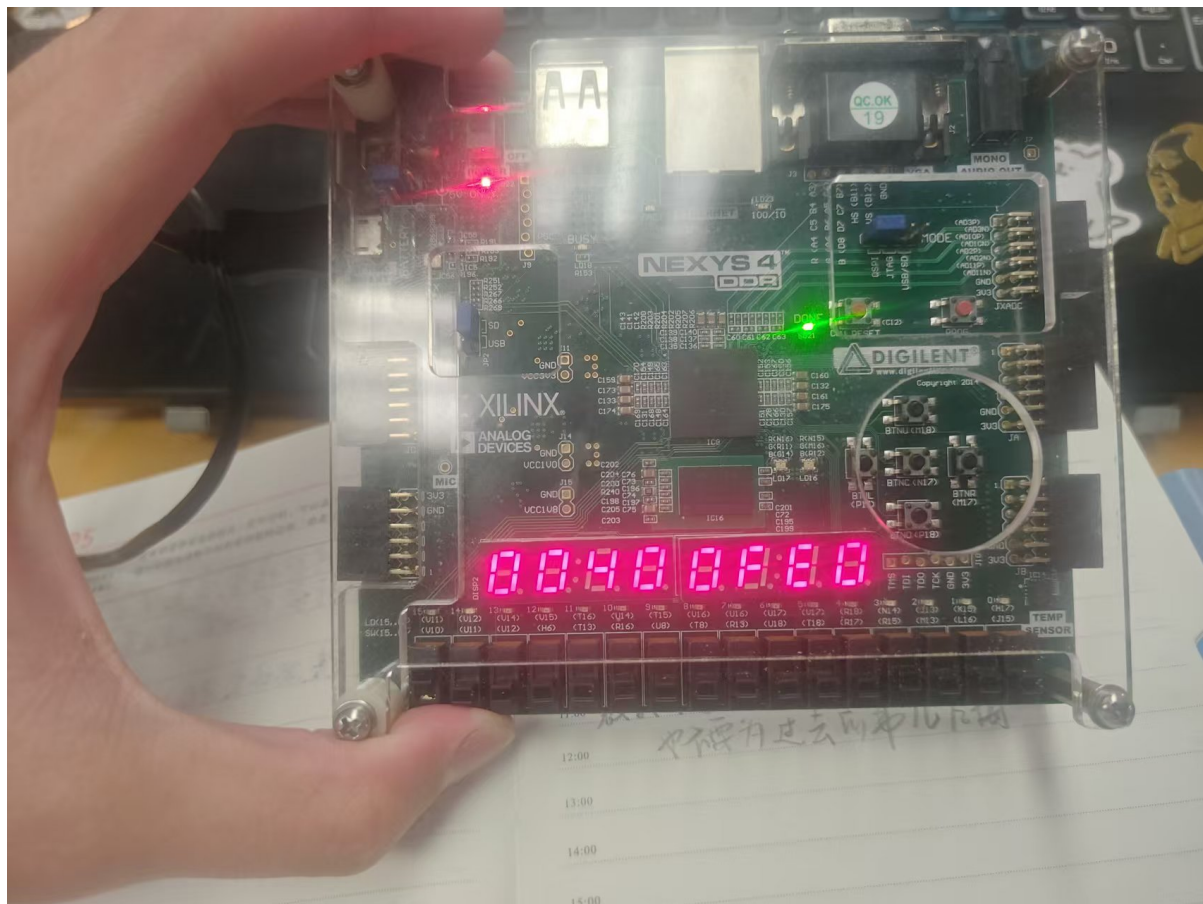


图 5.3 board

在下板过程中，我发现一个极为严重的问题，有数百个 critical warning 让我没有办法进行 implementation。critical 的详细报错的内容具体表述的意义是：两个驱动同时驱动了同一个端口。这个错误是在我之前进行后仿真的时候就存在的，因为程序可以正常运行，就被我忽略了。但是现在我不得不着手解决这个 critical warning。在顶层模块与 cpu 当中，我分别实例化了一个 IMEM 和 DMEM。产生这样问题的原因是我最初直接使用了 cpu 作为 top 模块，然后根据上传要求把原来的 top 改成了 cpu，并新建了一个新的 cpu。但是当时忘记删除了这个 cpu 中的 imem 模块，不仅影响了我的时序逻辑报告，同时影响了后仿真的错误排查。不过删除旧的多余模块就可以运行了。最后我的时钟频率调整到了 40MHz。

6 心得体会及建议

6.1 到底如何从零开始学习 cpu

我最初对于 cpu 的了解并不是从数字逻辑课程开始的，我的了解开始于一款游戏，名叫 turing complete，翻译过来的名字叫做图灵完备，在去年学习汇编语言的时候我就开始玩这个游戏了。显然的，这就是一款完成 cpu 的游戏。游戏大致模仿 scratch 语言来要求玩家从基础的与非门一个个搭建出完备的逻辑门架构，再搭建时钟，锁存器等等，最终搭建出一个完整的 cpu。我在这个软件当中完成了我的第一个八位八指令组合逻辑 cpu。我的 cpu 当时只有条件跳转，加减等等最基本的指令。我的 regfile 只有四个寄存器，甚至加法没有操作数，只有从一号寄存器和二号寄存器中取出加数将运算结果送到三号寄存器中这一条指令通路。但是我从此对于 cpu 有了基本的了解。

6.2 对于实验文档的一些改进建议

- 在指令说明的文档当中，加入汇编指令当中的寄存器与机器码当中的寄存器的对应关系，否则会导致设计 controller 的过程中产生指令混乱。例如，add 指令当中，\$1 \$2 \$3 并不是按照 rs rt rd 的顺序出现的。
- 应当将所有的实验文档整合成一个文件的多个部分，而非分散在多个文件夹当中，这样会对学生产生不必要的麻烦。

7 附录：工程所有代码

代码以及相关设计文档已经存放到 github 远程代码库 <https://github.com/Ainski/tongjicpu31/>

|
|
|
|
|
|
|
|
|
|
装
|
|
|
|
|
订
|
|
|
|
|
线
|
|
|
|
|
|
|
|
|
|
|

Listing 4 ALU.v

```

1  `timescale 1ns / 1ps
2  module alu(
3  input [31:0] a,    //32  F2  1
4  input [31:0] b,    //32  F2  2
5  input [3:0] aluc, //4   F2  alu p
6  output reg [31:0] r, //32  a; b%1 aluc3 p
7  output reg zero,
8  output reg carry,
9  //0 ±
10 // % ±
11 output reg negative, // ,0 ±
12 output reg overflow //
13 );
begin
14  always @ (*) begin
15      case (aluc)
16          4'b0000://  °z  "
17          begin
18              r = a + b;
19              zero = (r == 0);
20              carry = (a[31] & b[31]) | (a[31] & ~r[31]) | (b[31] & ~r[31]);
21              negative = (r[31] == 1);
22              overflow = 0;
23          end
24          4'b0010://  °z  "
25          begin
26              r = $signed(a) + $signed(b);
27              zero = (r == 0);
28              carry = 0;
29              negative = (r[31] == 1);
30              overflow = (a[31] & b[31] & ~r[31]) | (~a[31] & ~b[31] & r[31]);
31          end
32          4'b0001://  °z
33          begin
34              r = a - b;
35              zero = (r == 0);
36              carry = (~a[31] & b[31]) | (~a[31] & r[31]) | (~b[31] & r[31]);
37              negative = (r[31] == 1);
38              overflow = 0;
39          end

```

```

40      4'b0011://  0z
41  begin      r = $signed(a) - $signed(b);
42      zero = (r == 0);
43      carry = 0;
44      negative = (r[31] == 1);
45      overflow = (~a[31] & b[31] & r[31]) | (a[31] & ~b[31] & ~r[31]);
46  end
47  4'b0100://
48  begin
49      r = a & b;
50      zero = (r == 0);
51      carry = 0;
52      negative = (r[31] == 1);
53      overflow = 0;
54  end
55  4'b0101://>
56  begin
57      r = a | b;
58      zero = (r == 0);
59      carry = 0;
60      negative = (r[31] == 1);
61      overflow = 0;
62  end
63  4'b0110://
64  begin
65      r = a ^ b;
66      zero = (r == 0);
67      carry = 0;
68      negative = (r[31] == 1);
69      overflow = 0;
70  end
71  4'b0111://nor
72  begin
73      r = ~(a | b);
74      zero = (r == 0);
75      carry = 0;
76      negative = (r[31] == 1);
77      overflow = 0;
78  end
79  4'b1000://Lui

```

```

80     begin
81     r={b[15:0],16'b0};      zero = (r==0);
82         carry = 0;
83         negative = (r[31] == 1);
84         overflow = 0;
85     end
86     4'b1001://Lui
87     begin
88         r={b[15:0],16'b0};
89         zero = (r==0);
90         carry = 0;
91         negative = (r[31] == 1);
92         overflow = 0;
93     end
94     4'b1011://Slt
95     begin
96         r = ($signed(a) < $signed(b));
97         zero = (($signed(a) - $signed(b)) == 0);
98         carry = 0;
99         negative = ($signed(a) < $signed(b));
100        overflow = 0;
101    end
102    4'b1010://Sltu
103    begin
104        r = (a < b);
105        zero = ((a-b) == 0);
106        carry = (a<b);
107        negative = (r[31] == 1);
108        overflow = 0;
109    end
110    4'b1100://Sra
111    begin
112        r=($signed(b) >>> $signed(a));
113        zero = (r == 0);
114        //carryI      ~  »      p
115        if(a<32&&~a>0)
116            carry = b[a];
117        else if(a==0)
118            carry = 0;
119        else

```

```

120         carry=b[31];
121 negative = (r[31] == 1);      overflow = 0;
122     end
123     4'b1101://srl
124     begin
125         r=b>>a;
126         zero = (r == 0);
127         //carryI      ' »      µ
128         if(a<32&&a>0)
129             carry = b[a];
130         else
131             carry = 0;
132             negative = (r[31] == 1);
133             overflow = 0;
134     end
135     4'b1110://sll
136     begin
137         r=b<<a;
138         zero = (r == 0);
139         //carryI      ' »      µ
140         if(a<32&&a>0)
141             carry=b[32-a];
142         else
143             carry=0;
144             negative = (r[31] == 1);
145             overflow = 0;
146     end
147     4'b1111://sla
148     begin
149         r=b<<a;
150         zero = (r == 0);
151         //carryI      ' »      µ
152         carry = b[31];
153         negative = (r[31] == 1);
154         overflow = 0;
155     end
156 endcase
157 end
158 endmodule

```

Listing 5 controller.v

```

1  `timescale 1ns / 1ps
2  module controller(
3      input [5:0] op,
4      input [5:0] func,
5      output [1:0] M1,
6      output [1:0] M2,
7      output [1:0] M3,
8      output M4,
9      output [1:0] M5,
10     output M6,
11     output IM_R,
12     output RF_W,
13     output su,
parameter 14     output [3:0] aluc,
15     output CS,
16     output DM_W,
17     output DM_R,
18     output Btype
19 );
parameter 20     parameter add = 6'b100000;
21     parameter addu = 6'b100001;
22     parameter sub = 6'b100010;
23     parameter subu = 6'b100011;
24     parameter and_op = 6'b100100;
25     parameter or_op = 6'b100101;
26     parameter xor_op = 6'b100110;
parameter 27     parameter nor_op = 6'b100111;
28     parameter slt = 6'b101010;
29     parameter sltu = 6'b101011;
30     parameter sll = 6'b000000;
31     parameter srl = 6'b000010;
32     parameter sra = 6'b000011;
33     parameter sllv = 6'b000100;
34     parameter srlv = 6'b000110;
35     parameter srav = 6'b000111;
36     parameter jr = 6'b001000;
37
38     parameter addi = 6'b001000;
39     parameter addiu = 6'b001001;

```

```

40     parameter andi=6'b001100;
41 parameter ori=6'b001101;     parameter xori=6'b001110;
42     parameter lw=6'b100011;
43     parameter sw=6'b101011;
44     parameter beq=6'b000100;
45     parameter bne=6'b000101;
46     parameter slti=6'b001010;
47     parameter sltiu=6'b001011;
48     parameter lui=6'b001111;
49     parameter j=6'b000010;
50     parameter jal=6'b000011;
51
52 // add 0010
53 // addu 0000
41 wire 54 // sub 0011
55 // subu 0001
56 // and 0100
57 // or 0101
58 // xor 0110
59 // nor 0111
41 wire 60 // slt 1011
61 // sltu 1010
62 // sll 1110
63 // srl 1101
64 // sra 1100
65 // lui 1000
66     parameter add_aluc=4'b0010;
41 wire 67     parameter addu_aluc=4'b0000;
68     parameter sub_aluc=4'b0011;
69     parameter subu_aluc=4'b0001;
70     parameter and_aluc=4'b0100;
71     parameter or_aluc=4'b0101;
72     parameter xor_aluc=4'b0110;
73     parameter nor_aluc=4'b0111;
74     parameter slt_aluc=4'b1011;
75     parameter sltu_aluc=4'b1010;
76     parameter sll_aluc=4'b1110;
77     parameter srl_aluc=4'b1101;
78     parameter sra_aluc=4'b1100;
79     parameter lui_aluc=4'b1000;

```

```

80
81 wire addT, adduT, subT, subuT, andT, orT, xorT,          norT, sltT, sltuT,  sllT,  srlT,
    sraT, sllvT,
82     srlvT, sravT, jrT,  addiT, addiuT, andiT,  oriT,
83     xoriT, lwT,  swT,  beqT, bneT, sltiT, sltiuT,
84     luiT, jT, jalT;
85 assign addT = (op==0) & (func==add);
86 assign adduT = (op==0) & (func==addu);
87 assign subT = (op==0) & (func==sub);
88 assign subuT = (op==0) & (func==subu);
89 assign andT = (op==0) & (func==and_op);
90 assign orT = (op==0) & (func==or_op);
91 assign xorT = (op==0) & (func==xor_op);
92 assign norT = (op==0) & (func==nor_op);
assign 93 assign sltT = (op==0) & (func==slt);
94 assign sltuT = (op==0) & (func==sltu);
95 assign sllT = (op==0) & (func==sll);
96 assign srlT = (op==0) & (func==srl);
97 assign sraT = (op==0) & (func==sra);
98 assign sllvT = (op==0) & (func==sllv);
assign 99 assign srlvT = (op==0) & (func==srlv);
assign 100 assign sravT = (op==0) & (func==srav);
101 assign jrT = (op==0) & (func==jr);
102
103 assign addiT=(op==addi);
104 assign addiuT=(op==addiu);
105 assign andiT=(op==andi);
assign 106 assign oriT=(op==ori);
107 assign xoriT=(op==xori);
108 assign lwT=(op==lw);
109 assign swT=(op==sw);
110 assign beqT=(op==beq);
111 assign bneT=(op==bne);
112 assign sltiT=(op==slti);
113 assign sltiuT=(op==sltiu);
114 assign luiT=(op==lui);
115 assign jT=(op==j);
116 assign jalT=(op==jal);
117
118 assign M1=(jrT)?2'b10:(jT||jalT)?2'b01:2'b00;

```



```

119     assign IM_R=1;
120 assign RF_W=(jrT|swT|beqT|bneT|jT)?0:1;    assign su=(addiT|addiuT|lwT|swT|beqT|bneT|sltiT)?1:0;
121     assign M2=(lwT)?2'b01:(jalT)?2'b10:2'b00;
122     assign M4=(addiT|addiuT|andiT|oriT|xoriT|lwT|swT|sltiT|sltiuT|luiT)?1:0;
123     assign CS=(lwT|swT)?1:0;
124     assign DM_W=(swT)?1:0;
125     assign DM_R=(lwT)?1:0;
126     assign Btype=(beqT|bneT)?1:0;
127     assign aluc=
128     (addT|addiT)?add_aluc:
129     (adduT|addiuT)?addu_aluc:
130     (subT|subuT|bneT|beqT)?sub_aluc:
131     (andT|andiT)?and_aluc:
132     (orT|oriT)?or_aluc:
133     (xorT|xoriT)?xor_aluc:
134     (norT)?nor_aluc:
135     (sltT|sltiT)?slt_aluc:
136     (sltuT|sltiuT)?sltu_aluc:
137     (sllT|sllvT)?sll_aluc:
138     (srlT|srlvT)?srl_aluc:
139     (sraT|sravT)?sra_aluc:
140     (luiT)?lui_aluc:0;
141     assign M5=(sllT|srlT|sraT)?2'b01:(luiT)?2'b10:2'b00;
142     assign M6=(beqT)?1:0;
143     assign M3=(lwT|swT|sltiT|sltiuT|luiT|addiT|addiuT|andiT|oriT|xoriT)?2'b10:(jalT)?2'b01:2'b00;
144 endmodule

```

Listing 6 cpu.v

```

1  `timescale 1ns / 1ps
2  module cpu(
3      input clk,
4      input reset,
5      input [31:0] instr,
6      input [5:0] op,
7      input [4:0] rsc,
8      input [4:0] rtc,
9      input [4:0] rdc,
10     input [4:0] shamT,
11     input [5:0] func,
12     input [15:0] imdt, //immediate
13     input [25:0] index,
output 14     input [31:0] DMEdata,
15     output [31:0] mux1out,      // 保留output声明
16     output [31:0] NPCout,
17     output [31:0] a,
18     output [31:0] b,
19     output [31:0] imdt,
output 20     output [31:0] jextend,
21     output [4:0] mux3out,
22     output [31:0] npc,
23     output [31:0] pc,
24     output [31:0] rd,
25     output [31:0] rdd,
output 26     output [31:0] r,
27     output [31:0] rs,
28     output [31:0] rt,
29     output [31:0] shamT,
30     output [3:0] aluc,
31     output [3:0] jpc,
32     output [1:0] M1,
33     output [1:0] M2,
34     output [1:0] M3,
35     output M4,
36     output [1:0] M5,
37     output Btype,
38     output carry,
39     output CS,

```

```

40     output DM_R,
41 output DM_W,    output IM_R,
42     output M6,
43     output negative,
44     output overflow,
45     output RF_CLK,
46     output RF_W,
47     output su,
48     output zero,
49     output Ze,
50     output [31:0] regfile0,regfile1,regfile2,regfile3,regfile4,regfile5,regfile6,regfile7,
51     regfile8,regfile9,regfile10,regfile11,regfile12,regfile13,regfile14,regfile15,
52     regfile16,regfile17,regfile18,regfile19,regfile20,regfile21,regfile22,regfile23,
53     regfile24,regfile25,regfile26,regfile27,regfile28,regfile29,regfile30,regfile31
54 );
wire
55
56 // 声明内部wire信号 (原output变为wire并加_t后缀)
57 wire [31:0] mux1out_t;
58 wire [31:0] NPCout_t;
59 wire [31:0] a_t;
60 wire [31:0] b_t;
wire
61 wire [31:0] imdt_t;
62 wire [31:0] jextend_t;
63 wire [4:0] mux3out_t;
64 wire [31:0] npc_t;
65 wire [31:0] pc_t;
66 wire [31:0] rd_t;
wire
67 wire [31:0] rdd_t;
68 wire [31:0] r_t;
69 wire [31:0] rs_t;
70 wire [31:0] rt_t;
71 wire [31:0] shamt_t;
72 wire [3:0] aluc_t;
73 wire [3:0] jpc_t;
74 wire [1:0] M1_t;
75 wire [1:0] M2_t;
76 wire [1:0] M3_t;
77 wire M4_t;
78 wire [1:0] M5_t;
79 wire Btype_t;

```

```

80 wire carry_t;
81 wire CS_t; wire DM_R_t;
82 wire DM_W_t;
83 wire IM_R_t;
84 wire M6_t;
85 wire negative_t;
86 wire overflow_t;
87 wire RF_CLK_t;
88 wire RF_W_t;
89 wire su_t;
90 wire zero_t;
91 wire Ze_t;
92 wire [31:0] regfile0_t, regfile1_t, regfile2_t, regfile3_t, regfile4_t, regfile5_t, regfile6_t, regfile7_t,
93             regfile8_t, regfile9_t, regfile10_t, regfile11_t, regfile12_t, regfile13_t, regfile14_t,
assign      regfile15_t,
94             regfile16_t, regfile17_t, regfile18_t, regfile19_t, regfile20_t, regfile21_t, regfile22_t,
             regfile23_t,
95             regfile24_t, regfile25_t, regfile26_t, regfile27_t, regfile28_t, regfile29_t, regfile30_t,
             regfile31_t;
96
97 // 将内部wire信号赋值给模块输出端口
assign 98 assign mux1out = mux1out_t;
99 assign NPCout = NPCout_t;
100 assign a = a_t;
101 assign b = b_t;
102 assign imdt = imdt_t;
103 assign jextend = jextend_t;
assign 104 assign mux3out = mux3out_t;
105 assign npc = npc_t;
106 assign pc = pc_t;
107 assign rd = rd_t;
108 assign rdd = rdd_t;
109 assign r = r_t;
110 assign rs = rs_t;
111 assign rt = rt_t;
112 assign shamt = shamt_t;
113 assign aluc = aluc_t;
114 assign jpc = jpc_t;
115 assign M1 = M1_t;
116 assign M2 = M2_t;

```

```

117 assign M3 = M3_t;
118 assign M4 = M4_t; assign M5 = M5_t;
119 assign Btype = Btype_t;
120 assign carry = carry_t;
| 121 assign CS = CS_t;
| 122 assign DM_R = DM_R_t;
| 123 assign DM_W = DM_W_t;
| 124 assign IM_R = IM_R_t;
| 125 assign M6 = M6_t;
| 126 assign negative = negative_t;
| 127 assign overflow = overflow_t;
| 128 assign RF_CLK = RF_CLK_t;
| 129 assign RF_W = RF_W_t;
| 130 assign su = su_t;
| 131 assign zero = zero_t;
assign 132 assign Ze = Ze_t;
| 133 assign regfile0 = regfile0_t;
| 134 assign regfile1 = regfile1_t;
| 135 assign regfile2 = regfile2_t;
| 136 assign regfile3 = regfile3_t;
| 137 assign regfile4 = regfile4_t;
assign 138 assign regfile5 = regfile5_t;
| 139 assign regfile6 = regfile6_t;
| 140 assign regfile7 = regfile7_t;
| 141 assign regfile8 = regfile8_t;
| 142 assign regfile9 = regfile9_t;
| 143 assign regfile10 = regfile10_t;
assign 144 assign regfile11 = regfile11_t;
| 145 assign regfile12 = regfile12_t;
| 146 assign regfile13 = regfile13_t;
| 147 assign regfile14 = regfile14_t;
| 148 assign regfile15 = regfile15_t;
| 149 assign regfile16 = regfile16_t;
| 150 assign regfile17 = regfile17_t;
| 151 assign regfile18 = regfile18_t;
| 152 assign regfile19 = regfile19_t;
| 153 assign regfile20 = regfile20_t;
| 154 assign regfile21 = regfile21_t;
| 155 assign regfile22 = regfile22_t;
156 assign regfile23 = regfile23_t;

```

```

157 assign regfile24 = regfile24_t;
158 assign regfile25 = regfile25_t; assign regfile26 = regfile26_t;
159 assign regfile27 = regfile27_t;
160 assign regfile28 = regfile28_t;
161 assign regfile29 = regfile29_t;
162 assign regfile30 = regfile30_t;
163 assign regfile31 = regfile31_t;
164
165 // 实例化模块并连接内部wire信号 (后缀_t)
166 alu alu_inst(
167     .a(a_t),
168     .aluc(aluc_t),
169     .b(b_t),
170     .carry(carry_t),
171     .negative(negative_t),
172     .overflow(overflow_t),
173     .r(r_t),
174     .zero(zero_t)
175 );
176
177 controller controller_inst(
178     .Btype(Btype_t),
179     .DM_R(DM_R_t),
180     .DM_W(DM_W_t),
181     .IM_R(IM_R_t),
182     .M1(M1_t),
183     .M2(M2_t),
184     .M3(M3_t),
185     .M4(M4_t),
186     .M5(M5_t),
187     .M6(M6_t),
188     .RF_W(RF_W_t),
189     .aluc(aluc_t),
190     .func(func),
191     .op(op),
192     .CS(CS_t),
193     .su(su_t)
194 );
195
196 DMEM dmem_inst(

```

```

197     .CS(CS_t),
198     .DM_R(DM_R_t),    .DM_W(DM_W_t),
199     .DMEMaddr(r_t),
200     .DMEMdata(DMEMdata),
| 201     .clk(clk),
| 202     .rt(rt_t)
| 203 );
| 204
| 205 imdt_ext imdt_ext_inst(
| 206     .imdt(imdt_t),
| 207     .imdtT(imdtT),
| 208     .su(su_t)
| 209 );
| 210
| 211 IMEM imem_inst(
mux3 | 212     .IM_R(IM_R_t),
| 213     .address(pc_t),
| 214     .func(func),
| 215     .index(index),
| 216     .instr(instr),
| 217     .imdtT(imdtT),
mux3 | 218     .op(op),
| 219     .rdc(rdc),
| 220     .rsc(rsc),
| 221     .rtc(rtc),
| 222     .shamtT(shamtT)
| 223 );
mux3 | 224
| 225 jextend jextend_inst(
| 226     .index(index),
| 227     .jextend(jextend_t),
| 228     .jpc(jpc_t)
| 229 );
| 230
| 231 mux1 mux1_inst(
| 232     .M1(M1_t),
| 233     .NPCOut(NPCOut_t),
| 234     .jextend(jextend_t),
| 235     .mux1out(mux1out_t), // 模块内部连接使用_t信号
| 236     .rs(rs_t)

```

```

237 );
238 mux3 mux3_inst(    .M3(M3_t),
239     .rdc(rdc),
240     .rtc(rtc),
241     .mux3out(mux3out_t)
242 );
243
244 mux4 mux4_inst(
245     .M4(M4_t),
246     .imdt(imdt_t),
247     .rt(rt_t),
248     .b(b_t)
249 );
250
251 mux5 mux5_inst(
252     .M5(M5_t),
253     .shamt(shamt_t),
254     .rt(rt_t),
255     .rs(rs_t),
256     .a(a_t)
257 );
258
259 mux6 mux6_inst(
260     .M6(M6_t),
261     .Ze(Ze_t),
262     .zero(zero_t)
263 );
264
265 npcmaker npcmaker_inst(
266     .Btype(Btype_t),
267     .Ze(Ze_t),
268     .imdt(imdt_t),
269     .npc(npc_t), // 注意：原模块端口为npc，此处连接npc_t
270     .npc_out(NPCout_t)
271 );
272
273 pcreg pc_inst(
274     .jpc(jpc_t),
275     .mux1out(mux1out_t),
276     .npc(npc_t), // 连接npc_t

```



```

277     .pc(pc_t),
278     .pc_clk(clk),    .reset(reset)
279 );
280
281 regfile cpu_ref(
282     .RF_CLK(clk),
283     .RF_RST(reset),
284     .RF_W(RF_W_t),
285     .mux3out(mux3out_t),
286     .rd(rd_t),
287     .rdd(rdd_t),
288     .rsc(rsc),
289     .rs(rs_t),
290     .rtc(rtc),
291     .rt(rt_t),
292     .regfile0(regfile0_t),
293     .regfile1(regfile1_t),
294     .regfile2(regfile2_t),
295     .regfile3(regfile3_t),
296     .regfile4(regfile4_t),
297     .regfile5(regfile5_t),
298     .regfile6(regfile6_t),
299     .regfile7(regfile7_t),
300     .regfile8(regfile8_t),
301     .regfile9(regfile9_t),
302     .regfile10(regfile10_t),
303     .regfile11(regfile11_t),
304     .regfile12(regfile12_t),
305     .regfile13(regfile13_t),
306     .regfile14(regfile14_t),
307     .regfile15(regfile15_t),
308     .regfile16(regfile16_t),
309     .regfile17(regfile17_t),
310     .regfile18(regfile18_t),
311     .regfile19(regfile19_t),
312     .regfile20(regfile20_t),
313     .regfile21(regfile21_t),
314     .regfile22(regfile22_t),
315     .regfile23(regfile23_t),
316     .regfile24(regfile24_t),

```

```

317     .regfile25(regfile25_t),
318 .regfile26(regfile26_t),    .regfile27(regfile27_t),
319     .regfile28(regfile28_t),
320     .regfile29(regfile29_t),
321     .regfile30(regfile30_t),
322     .regfile31(regfile31_t)
323 );
324
325 shamt_ext shamt_ext_inst(
326     .shamt(shamt_t),
327     .shamtT(shamtT)
328 );
329
330 mux2 mux2_inst(
331     .M2(M2_t),
332     .NPC(npc_t), // 原模块端口为NPC，此处连接npc_t（注意信号名是否与模块定义一致）
333     .r(r_t),
334     .dmemdata(DMEMdata),
335     .rdd(rdd_t)
336 );
337
338 endmodule

```

Listing 7 DMEM.v

```

1  `timescale 1ns / 1ps
2  module DMEM(
3      input clk, // 时钟
4      input [31:0] rt,
5      input [31:0] DMEMaddr, // 地址
6      input CS,
7      input DM_W,
8      input DM_R,
9      output [31:0] DMEMdata
10 );
11
12 reg [31:0] DMEM [0:2047];
13 //initial begin
14 //    $readmemb("DMEM.txt", DMEM);
15 //end
16 always @(posedge clk) begin
17     if(CS) begin
18         if(DM_W) begin
19             DMEM[DMEMaddr[12:2]] = rt;
20         end
21     end
22 end
23 //assign DMEMdata = DMEM[DMEMaddr]&CS&DM_R;
24 assign DMEMdata = (CS&&DM_R)?DMEM[DMEMaddr[12:2]]:31'b0;
25
26
27 endmodule

```

Listing 8 **gate.v**

```
1  `timescale 1ns / 1ps
2  module gate(input a,input ctrl);
3      assign a = ctrl? 1'b1 : 1'b0;
4  endmodule
```

装

订

线

Listing 9 imdtext.v

```

1  `timescale 1ns / 1ps
2  module imdt_ext(
3      input [15:0] imdtT,
4      input su,
5      output [31:0] imdt
6  );
7  assign imdt = su ? {{16{imdtT[15]}}, imdtT} : {16'b0, imdtT};
8
9  endmodule

```

Listing 10 imem.v

```

1 // (c) Copyright 1995-2025 Xilinx, Inc. All rights reserved.
2 //
3 // This file contains confidential and proprietary information
4 // of Xilinx, Inc. and is protected under U.S. and
5 // international copyright and other intellectual property
6 // laws.
7 //
8 // DISCLAIMER
9 // This disclaimer is not a license and does not grant any
10 // rights to the materials distributed herewith. Except as
11 // otherwise provided in a valid license issued to you by
12 // Xilinx, and to the maximum extent permitted by applicable
13 // law: (1) THESE MATERIALS ARE MADE AVAILABLE "AS IS" AND
14 // WITH ALL FAULTS, AND XILINX HEREBY DISCLAIMS ALL WARRANTIES
15 // AND CONDITIONS, EXPRESS, IMPLIED, OR STATUTORY, INCLUDING
16 // BUT NOT LIMITED TO WARRANTIES OF MERCHANTABILITY, NON-
17 // INFRINGEMENT, OR FITNESS FOR ANY PARTICULAR PURPOSE; and
18 // (2) Xilinx shall not be liable (whether in contract or tort,
19 // including negligence, or under any other theory of
20 // liability) for any loss or damage of any kind or nature
21 // related to, arising under or in connection with these
22 // materials, including for any direct, or any indirect,
23 // special, incidental, or consequential loss or damage
24 // (including loss of data, profits, goodwill, or any type of
25 // loss or damage suffered as a result of any action brought
26 // by a third party) even if such damage or loss was
27 // reasonably foreseeable or Xilinx had been advised of the
28 // possibility of the same.
29 //
30 // CRITICAL APPLICATIONS
31 // Xilinx products are not designed or intended to be fail-
32 // safe, or for use in any application requiring fail-safe
33 // performance, such as life-support or safety devices or
34 // systems, Class III medical devices, nuclear facilities,
35 // applications related to the deployment of airbags, or any
36 // other applications that could lead to death, personal
37 // injury, or severe property or environmental damage
38 // (individually and collectively, "Critical
39 // Applications"). Customer assumes the sole risk and

```

```

40 // liability of any use of Xilinx products in Critical
41 // Applications, subject only to applicable laws and// regulations governing limitations on product
    liability.
42 //
43 // THIS COPYRIGHT NOTICE AND DISCLAIMER MUST BE RETAINED AS
44 // PART OF THIS FILE AT ALL TIMES.
45 //
46 // DO NOT MODIFY THIS FILE.
47
48
49 // IP VLN: xilinx.com:ip:dist_mem_gen:8.0
50 // IP Revision: 10
51
52 `timescale 1ns/1ps
53
54 (* DowngradeIPIdentifiedWarnings = "yes" *)
55 module imem (
56     a,
57     spo
58 );
59
60 input wire [10 : 0] a;
61 output wire [31 : 0] spo;
62
63     dist_mem_gen_v8_0_10 #(
64         .C_FAMILY("artix7"),
65         .C_ADDR_WIDTH(11),
66         .C_DEFAULT_DATA("0"),
67         .C_DEPTH(2048),
68         .C_HAS_CLK(0),
69         .C_HAS_D(0),
70         .C_HAS_DPO(0),
71         .C_HAS_DPRA(0),
72         .C_HAS_I_CE(0),
73         .C_HAS_QDPO(0),
74         .C_HAS_QDPO_CE(0),
75         .C_HAS_QDPO_CLK(0),
76         .C_HAS_QDPO_RST(0),
77         .C_HAS_QDPO_SRST(0),
78         .C_HAS_QSPO(0),

```

```

79     .C_HAS_QSPO_CE(0),
80 .C_HAS_QSPO_RST(0),    .C_HAS_QSPO_SRST(0),
81     .C_HAS_SPO(1),
82     .C_HAS_WE(0),
83     .C_MEM_INIT_FILE("imem.mif"),
84     .C_ELABORATION_DIR("./"),
85     .C_MEM_TYPE(0),
86     .C_PIPELINE_STAGES(0),
87     .C_QCE_JOINED(0),
88     .C_QUALIFY_WE(0),
89     .C_READ_MIF(1),
90     .C_REG_A_D_INPUTS(0),
91     .C_REG_DPRA_INPUT(0),
92     .C_SYNC_ENABLE(1),
93     .C_WIDTH(32),
94     .C_PARSER_TYPE(1)
95 ) inst (
96     .a(a),
97     .d(32'B0),
98     .dpra(11'B0),
99     .clk(1'D0),
100    .we(1'D0),
101    .i_ce(1'D1),
102    .qspo_ce(1'D1),
103    .qdpo_ce(1'D1),
104    .qdpo_clk(1'D0),
105    .qspo_rst(1'D0),
106    .qdpo_rst(1'D0),
107    .qspo_srst(1'D0),
108    .qdpo_srst(1'D0),
109    .spo(spo),
110    .dpo(),
111    .qspo(),
112    .qdpo()
113 );
114 endmodule

```


Listing 11 IMEMtop.v

```

1  `timescale 1ns / 1ps
2  module IMEM(
3      input [31:0] address,
4      input IM_R, //read a intruction
5      output reg [31:0] instr,
6      output [5:0] op,
7      output [4:0] rsc,
8      output [4:0] rtc,
9      output [4:0] rdc,
10     output [4:0] shamtT,
11     output [5:0] func,
12     output [15:0] imdtT, //immediate
13     output [25:0] index
14
15 );
16     assign op=instr[31:26];
17     assign rsc=instr[25:21];
18     assign rtc=instr[20:16];
19     assign rdc=instr[15:11];
20     assign shamtT=instr[10:6];
21     assign func=instr[5:0];
22     assign imdtT=instr[15:0];
23     assign index=instr[25:0];
24     wire [31:0] instrT;
25     always @(*) begin
26         instr=IM_R?instrT : 32'h0;
27     end
28     imem imem_ip(
29         .a(address[12:2]),
30         .spo(instrT)
31     );
32 //     reg [31:0] IMEM [0:2047];
33 //     initial begin
34 //         // $readmemb("E:/github/cpu31/test_datas/_1_addi.txt", IMEM);
35 //         //$readmemb("E:/github/cpu31/test_datas/_1_addiu.txt", IMEM);
36 //         //$readmemb("E:/github/cpu31/test_datas/_1_lui.txt", IMEM);
37 //         //$readmemb("E:/github/cpu31/test_datas/_2_add.txt", IMEM);
38 //         //$readmemb("E:/github/cpu31/test_datas/_2_addu.txt", IMEM);
39 //         //$readmemb("E:/github/cpu31/test_datas/_2_and.txt", IMEM);

```

```

40 //      //$readmemb("E:/github/cpu31/test_datas/_2_andi.txt", IMEM);
41 //      //$readmemb("E:/github/cpu31/test_datas/_2_lsw.txt", IMEM);//      //$readmemb("E:/
    github/cpu31/test_datas/_2_lsw2.txt", IMEM);
42 //      //$readmemb("E:/github/cpu31/test_datas/_2_nor.txt", IMEM);
43 //      //$readmemb("E:/github/cpu31/test_datas/_2_or.txt", IMEM);
44 //      //$readmemb("E:/github/cpu31/test_datas/_2_ori.txt", IMEM);
45 //      //$readmemb("E:/github/cpu31/test_datas/_2_sll.txt", IMEM);
46 //      //$readmemb("E:/github/cpu31/test_datas/_2_sllv.txt", IMEM);
47 //      //$readmemb("E:/github/cpu31/test_datas/_2_slt.txt", IMEM);
48 //      //$readmemb("E:/github/cpu31/test_datas/_2_slti.txt", IMEM);
49 //      //$readmemb("E:/github/cpu31/test_datas/_2_sltiu.txt", IMEM);
50 //      //$readmemb("E:/github/cpu31/test_datas/_2_sltu.txt", IMEM);
51 //      //$readmemb("E:/github/cpu31/test_datas/_2_sra.txt", IMEM);
52 //      //$readmemb("E:/github/cpu31/test_datas/_2_srav.txt", IMEM);
53 //      //$readmemb("E:/github/cpu31/test_datas/_2_srl.txt", IMEM);
54 //      //$readmemb("E:/github/cpu31/test_datas/_2_srlv.txt", IMEM);
55 //      //$readmemb("E:/github/cpu31/test_datas/_2_sub.txt", IMEM);
56 //      //$readmemb("E:/github/cpu31/test_datas/_2_subu.txt", IMEM);
57 //      //$readmemb("E:/github/cpu31/test_datas/_2_xor.txt", IMEM);
58 //      //$readmemb("E:/github/cpu31/test_datas/_2_xori.txt", IMEM);
59 //      //$readmemb("E:/github/cpu31/test_datas/_3.5_beq.txt", IMEM);
60 //      //$readmemb("E:/github/cpu31/test_datas/_3.5_bne.txt", IMEM);
61 //      //$readmemb("E:/github/cpu31/test_datas/_3_j.txt", IMEM);
62 //      //$readmemb("E:/github/cpu31/test_datas/_3_jal.txt", IMEM);
63 //      //$readmemb("E:/github/cpu31/test_datas/_4_jr.txt", IMEM);
64
65 //      end
66
67
68 endmodule

```

Listing 12 Jextend.v

```

1  `timescale 1ns / 1ps
2  module jextend(
3      input [3:0] jpc,
4      input [25:0] index,
5      output [31:0] jextend
6  );
7      assign jextend={jpc, index[25:21],1'b0,index[19:0],2'b0};
8  endmodule

```

装

订

线

Listing 13 mux1.v

```

1  `timescale 1ns/1ps
2  module mux1 (
3      input wire [31:0] jextend,
4      input wire [31:0] NPCout,
5      input wire [31:0] rs,
6      input wire [1:0] M1,
7      output reg [31:0] mux1out
8  );
9
10 always @(*) begin
11     case (M1)
12         2'b00: mux1out = NPCout;
13         2'b01: mux1out = jextend;
14         2'b10: mux1out = rs - 32'h00400000;
15         default: mux1out = 32'b0;
16     endcase
17 end
18
19 endmodule

```

Listing 14 **mux2.v**

```

1  `timescale 1ns / 1ps
2  module mux2(
3      input [31:0] NPC,r,dmemdata,
4      input [1:0] M2,
5      output [31:0] rdd
6  );
7  assign rdd = (M2 == 2'b00)? r :
8      (M2 == 2'b01)? dmemdata :
9      (M2 == 2'b10)? NPC+32'h00400000 :
10     32'h0;
11 endmodule

```

装

订

线

Listing 15 mux3.v

```

1  `timescale 1ns / 1ps
2  module mux3(
3      input [4:0] rtc,
4      input [4:0] rdc,
5      input [1:0] M3,
6      output [4:0] mux3out
7  );
8      assign mux3out =
9          (M3 == 2'b00) ? rdc :      // rdc
10         (M3 == 2'b01) ? 5'b11111 : // 5'b11111
11         (M3 == 2'b10) ? rtc :      // rtc
12         5'b0;                      // M3=11 0
13 endmodule

```

Listing 16 **mux4.v**

```

1  `timescale 1ns / 1ps
2  module mux4(
3      input [31:0] rt,imdt,
4      input M4,
5      output [31:0] b);
6      assign b = M4? imdt : rt;
7  endmodule

```

装

订

线

Listing 17 mux5.v

```

1  `timescale 1ns / 1ps
2  module mux5(
3      input wire [31:0] rt,
4      input wire [31:0] rs,
5      input wire [31:0] shamt,
6      input wire [1:0] M5,
7      output wire [31:0] a
8  );
9      assign a =
10         (M5==2'b01)? shamt :
11         (M5 == 2'b00)? rs :
12         (M5 == 2'b10)? 32'b0 :
13         (M5==2'b11)? rt:32'b0;
14 endmodule

```


Listing 18 mux6.v

```

1  `timescale 1ns / 1ps
2  module mux6(
3      input zero,
4      input M6,
5      output Ze
6  );
7  assign Ze = !(zero ^ M6);
8  endmodule

```

装

订

线

Listing 19 npcmaker.v

```

1  `timescale 1ns / 1ps
2  module npcmaker(
3      input  [31:0] imdt,    // 指令字
4      input  [31:0] npc,    // 程序计数器
5      input          Btype,  // 分支类型
6      input          Ze,     // ALU 标志
7      output [31:0] npc_out // 新的程序计数器
8  );
9
10 // 计算新的程序计数器
11 // 如果分支类型不为0，则新的程序计数器为 npc + (imdt[29:0] << 2)
12 // 否则，新的程序计数器为 npc + 4
13
14 assign npc_out = (Ze && Btype) ? npc + {imdt[29:0], 2'b00} : npc;
15
16
17 endmodule

```

Listing 20 pc.v

```

1  `timescale 1ns/1ps
2  module pcreg(
3      input pc_clk,
4      input reset,          // 复位信号
5      input [31:0] mux1out,
6      output [31:0] npc,
7      output reg [31:0] pc,
8      output [3:0] jpc
9  );
10
11  assign jpc = pc[31:28]; // PC 高 4 位
12
13  // 当 reset 有效时，PC 初始化为 32'b0
14  always @(posedge pc_clk or posedge reset) begin
15      if (reset) begin // reset 有效时，PC
16          pc <= 32'b0;
17      end else begin // 否则，PC
18          pc <= mux1out;
19      end
20  end
21
22  assign npc = pc + 32'd4; // PC 加 4
23
24  endmodule

```

Listing 21 regfile.v

```

1  `timescale 1ns / 1ps
2  module regfile(
3      input RF_CLK,
4      input RF_RST,
5      input RF_W,
6      input [31:0] rdd,
7      input [4:0] mux3out,
8      input [4:0] rsc,
9      input [4:0] rtc,
10     output [31:0] rt,
11     output [31:0] rd,
12     output [31:0] rs,
13     output [31:0] regfile0,
14     output [31:0] regfile1,
15     output [31:0] regfile2,
16     output [31:0] regfile3,
17     output [31:0] regfile4,
18     output [31:0] regfile5,
19     output [31:0] regfile6,
20     output [31:0] regfile7,
21     output [31:0] regfile8,
22     output [31:0] regfile9,
23     output [31:0] regfile10,
24     output [31:0] regfile11,
25     output [31:0] regfile12,
26     output [31:0] regfile13,
27     output [31:0] regfile14,
28     output [31:0] regfile15,
29     output [31:0] regfile16,
30     output [31:0] regfile17,
31     output [31:0] regfile18,
32     output [31:0] regfile19,
33     output [31:0] regfile20,
34     output [31:0] regfile21,
35     output [31:0] regfile22,
36     output [31:0] regfile23,
37     output [31:0] regfile24,
38     output [31:0] regfile25,
39     output [31:0] regfile26,

```

```

40     output [31:0] regfile27,
41 output [31:0] regfile28,    output [31:0] regfile29,
42     output [31:0] regfile30,
43     output [31:0] regfile31
44 );
45
46 reg [31:0] array_reg[0:31];
47 assign regfile0 = array_reg[0];
48 assign regfile1 = array_reg[1];
49 assign regfile2 = array_reg[2];
50 assign regfile3 = array_reg[3];
51 assign regfile4 = array_reg[4];
52 assign regfile5 = array_reg[5];
53 assign regfile6 = array_reg[6];
always 54 assign regfile7 = array_reg[7];
55 assign regfile8 = array_reg[8];
56 assign regfile9 = array_reg[9];
57 assign regfile10 = array_reg[10];
58 assign regfile11 = array_reg[11];
59 assign regfile12 = array_reg[12];
always 60 assign regfile13 = array_reg[13];
61 assign regfile14 = array_reg[14];
62 assign regfile15 = array_reg[15];
63 assign regfile16 = array_reg[16];
64 assign regfile17 = array_reg[17];
65 assign regfile18 = array_reg[18];
66 assign regfile19 = array_reg[19];
always 67 assign regfile20 = array_reg[20];
68 assign regfile21 = array_reg[21];
69 assign regfile22 = array_reg[22];
70 assign regfile23 = array_reg[23];
71 assign regfile24 = array_reg[24];
72 assign regfile25 = array_reg[25];
73 assign regfile26 = array_reg[26];
74 assign regfile27 = array_reg[27];
75 assign regfile28 = array_reg[28];
76 assign regfile29 = array_reg[29];
77 assign regfile30 = array_reg[30];
78 assign regfile31 = array_reg[31];
79

```

```

80     integer i; //      » · ± -
81 always @(posedge RF_CLK or posedge RF_RST) begin          if (RF_RST) begin
82         for (i = 0; i < 32; i=i+1) begin
83             array_reg[i] = 32'h0; // , ' j
84         end
85     end else if (RF_W && mux3out != 6'b0) begin // ± j
86         array_reg[mux3out] = rdd;
87     end
88 end
89
90 //      ± 8'3 « ±
91 assign rt = array_reg[rtc];
92 assign rd = array_reg[mux3out];
93 assign rs = array_reg[rsc];
94
95 endmodule

```

Listing 22 shamtext.v

```

1  `timescale 1ns / 1ps
2  module shamt_ext(
3      input [4:0] shamtT,
4      output [31:0] shamt
5  );
6      //  ) 5 shamtT) I32 27 2 1
7      assign shamt = {27'b0, shamtT};
8  endmodule

```

Listing 23 top.v

```

1  `timescale 1ns / 1ps
2  module sccomp_dataflow(
3      input clk_in,
4      input reset,
5      output [31:0] mux1out,
6      output [31:0] NPCOut,
7      output [31:0] a,
8      output [31:0] b,
9      output [31:0] DMEMdata,
10     output [31:0] imdt,
11     output [25:0] index,
12     output [31:0] inst,
13     output [31:0] jextend,
14     output [4:0] mux3out,
15     output [31:0] npc,
16     output [31:0] pc,
17     output [31:0] rd,
18     output [31:0] rdd,
19     output [31:0] r,
20     output [31:0] rs,
21     output [31:0] rt,
22     output [31:0] shamT,
23     output [5:0] func,
24     output [5:0] op,
25     output [4:0] rdc,
26     output [4:0] rsc,
27     output [4:0] rtc,
28     output [4:0] shamT,
29     output [3:0] aluc,
30     output [3:0] jpc,
31     output [15:0] imdtT,
32     output [1:0] M1,
33     output [1:0] M2,
34     output [1:0] M3,
35     output M4,
36     output [1:0] M5,
37     output Btype,
38     output carry,
39     output CS,

```



```

40     output DM_R,
41 output DM_W,    output IM_R,
42     output M6,
43     output negative,
44     output overflow,
45     output RF_CLK,
46
47     output RF_W,
48     output su,
49     output zero,
50     output Ze,
51     output [31:0] regfile0,regfile1,regfile2,regfile3,regfile4,regfile5,regfile6,regfile7,
52     regfile8,regfile9,regfile10,regfile11,regfile12,regfile13,regfile14,regfile15,
53     regfile16,regfile17,regfile18,regfile19,regfile20,regfile21,regfile22,regfile23,
41 wire 54     regfile24,regfile25,regfile26,regfile27,regfile28,regfile29,regfile30,regfile31
55 );
56 wire [31:0]pc_temp;
57 assign pc=pc_temp+32'h00400000;
58 assign RF_CLK = clk_in;
59
60
41 wire 61 wire [31:0] NPCout_t;
62 wire [31:0] a_t;
63 wire [31:0] b_t;
64 wire [31:0] DMEdata_t;
65 wire [31:0] imdt_t;
66 wire [25:0] index_t;
41 wire 67 wire [31:0] inst_t;
68 wire [31:0] jextend_t;
69 wire [4:0] mux3out_t;
70 wire [31:0] npc_t;
71 wire [31:0] rd_t;
72 wire [31:0] rdd_t;
73 wire [31:0] r_t;
74 wire [31:0] rs_t;
75 wire [31:0] rt_t;
76 wire [31:0] sham_t;
77 wire [5:0] func_t;
78 wire [5:0] op_t;
79 wire [4:0] rdc_t;

```

```

80 wire [4:0] rsc_t;
81 wire [4:0] rtc_t;wire [4:0] shamT_t;
82 wire [3:0] aluc_t;
83 wire [3:0] jpc_t;
| 84 wire [15:0] imdtT_t;
| 85 wire [1:0] M1_t;
| 86 wire [1:0] M2_t;
| 87 wire [1:0] M3_t;
| 88 wire M4_t;
| 89 wire [1:0] M5_t;
| 90 wire Btype_t;
| 91 wire carry_t;
| 92 wire CS_t;
| 93 wire DM_R_t;
assign 94 wire DM_W_t;
| 95 wire IM_R_t;
| 96 wire M6_t;
| 97 wire negative_t;
| 98 wire overflow_t;
| 99 wire RF_CLK_t;
| 100 wire RF_W_t;
assign 101 wire su_t;
| 102 wire zero_t;
| 103 wire Ze_t;
| 104 wire [31:0] regfile0_t,regfile1_t,regfile2_t,regfile3_t,regfile4_t,regfile5_t,regfile6_t,regfile7_t,
| 105 regfile8_t,regfile9_t,regfile10_t,regfile11_t,regfile12_t,regfile13_t,regfile14_t,
| regfile15_t,
assign 106 regfile16_t,regfile17_t,regfile18_t,regfile19_t,regfile20_t,regfile21_t,regfile22_t,
| regfile23_t,
| 107 regfile24_t,regfile25_t,regfile26_t,regfile27_t,regfile28_t,regfile29_t,regfile30_t,
| regfile31_t;
| 108 wire [31:0]mux1out_t;
| 109 assign mux1out=mux1out_t;
| 110 assign NPCout = NPCout_t;
| 111 assign a = a_t;
| 112 assign b = b_t;
| 113 assign DMEMdata = DMEMdata_t;
| 114 assign imdt = imdt_t;
| 115 assign index = index_t;
116 assign inst = inst_t;

```

```

117 assign jextend = jextend_t;
118 assign mux3out = mux3out_t; assign npc = npc_t;
119 assign rd = rd_t;
120 assign rdd = rdd_t;
| 121 assign r = r_t;
| 122 assign rs = rs_t;
| 123 assign rt = rt_t;
| 124 assign shamt = shamt_t;
| 125 assign func = func_t;
| 126 assign op = op_t;
| 127 assign rdc = rdc_t;
| 128 assign rsc = rsc_t;
| 129 assign rtc = rtc_t;
| 130 assign shamtT = shamtT_t;
assign 131 assign aluc = aluc_t;
| 132 assign jpc = jpc_t;
| 133 assign imdtT = imdtT_t;
| 134 assign M1 = M1_t;
| 135 assign M2 = M2_t;
| 136 assign M3 = M3_t;
| 137 assign M4 = M4_t;
assign 138 assign M5 = M5_t;
| 139 assign Btype = Btype_t;
| 140 assign carry = carry_t;
| 141 assign CS = CS_t;
| 142 assign DM_R = DM_R_t;
| 143 assign DM_W = DM_W_t;
assign 144 assign IM_R = IM_R_t;
| 145 assign M6 = M6_t;
| 146 assign negative = negative_t;
| 147 assign overflow = overflow_t;
| 148 assign RF_CLK = RF_CLK_t; // 注意：原代码中RF_CLK直接赋?%为clk_in, 此处需根据逻辑调整
| 149 assign RF_W = RF_W_t;
| 150 assign su = su_t;
| 151 assign zero = zero_t;
| 152 assign Ze = Ze_t;
| 153 assign regfile0 = regfile0_t;
| 154 assign regfile1 = regfile1_t;
| 155 assign regfile2 = regfile2_t;
156 assign regfile3 = regfile3_t;

```

```

157 assign regfile4 = regfile4_t;
158 assign regfile5 = regfile5_t; assign regfile6 = regfile6_t;
159 assign regfile7 = regfile7_t;
160 assign regfile8 = regfile8_t;
161 assign regfile9 = regfile9_t;
162 assign regfile10 = regfile10_t;
163 assign regfile11 = regfile11_t;
164 assign regfile12 = regfile12_t;
165 assign regfile13 = regfile13_t;
166 assign regfile14 = regfile14_t;
167 assign regfile15 = regfile15_t;
168 assign regfile16 = regfile16_t;
169 assign regfile17 = regfile17_t;
170 assign regfile18 = regfile18_t;
171 assign regfile19 = regfile19_t;
172 assign regfile20 = regfile20_t;
173 assign regfile21 = regfile21_t;
174 assign regfile22 = regfile22_t;
175 assign regfile23 = regfile23_t;
176 assign regfile24 = regfile24_t;
177 assign regfile25 = regfile25_t;
178 assign regfile26 = regfile26_t;
179 assign regfile27 = regfile27_t;
180 assign regfile28 = regfile28_t;
181 assign regfile29 = regfile29_t;
182 assign regfile30 = regfile30_t;
183 assign regfile31 = regfile31_t;
184 cpu sccpu(
185     .clk(clk_in),
186     .NPCOut(NPCOut_t),
187     .a(a_t),
188     .b(b_t),
189     .mux1out(mux1out_t),
190     .DMEMdata(DMEMdata_t),
191     .imdt(imdt_t),
192     .index(index_t),
193     .instr(inst_t),
194     .jextend(jextend_t),
195     .mux3out(mux3out_t),
196     .npc(npc_t),

```

```

197         .pc(pc_temp),
198 .rd(rd_t),          .rdd(rdd_t),
199         .r(r_t),
200         .rs(rs_t),
201         .rt(rt_t),
202         .shamt(shamt_t),
203         .func(func_t),
204         .op(op_t),
205         .rdc(rdc_t),
206         .rsc(rsc_t),
207         .rtc(rtc_t),
208         .shamtT(shamtT_t),
209         .aluc(aluc_t),
210         .jpc(jpc_t),
211         .imdtT(imdtT_t),
212         .M1(M1_t),
213         .M2(M2_t),
214         .M3(M3_t),
215         .M4(M4_t),
216         .M5(M5_t),
217         .Btype(Btype_t),
218         .carry(carry_t),
219         .CS(CS_t),
220         .DM_R(DM_R_t),
221         .DM_W(DM_W_t),
222         .IM_R(IM_R_t),
223         .M6(M6_t),
224         .negative(negative_t),
225         .overflow(overflow_t),
226         .RF_CLK(RF_CLK_t),
227         .RF_W(RF_W_t),
228         .reset(reset),
229         .su(su_t),
230         .zero(zero_t),
231         .Ze(Ze_t),
232         .regfile0(regfile0_t),
233         .regfile1(regfile1_t),
234         .regfile2(regfile2_t),
235         .regfile3(regfile3_t),
236         .regfile4(regfile4_t),

```

```

237         .regfile5(regfile5_t),
238 .regfile6(regfile6_t),          .regfile7(regfile7_t),
239         .regfile8(regfile8_t),
240         .regfile9(regfile9_t),
241         .regfile10(regfile10_t),
242         .regfile11(regfile11_t),
243         .regfile12(regfile12_t),
244         .regfile13(regfile13_t),
245         .regfile14(regfile14_t),
246         .regfile15(regfile15_t),
247         .regfile16(regfile16_t),
248         .regfile17(regfile17_t),
249         .regfile18(regfile18_t),
250         .regfile19(regfile19_t),
251         .regfile20(regfile20_t),
252         .regfile21(regfile21_t),
253         .regfile22(regfile22_t),
254         .regfile23(regfile23_t),
255         .regfile24(regfile24_t),
256         .regfile25(regfile25_t),
257         .regfile26(regfile26_t),
258         .regfile27(regfile27_t),
259         .regfile28(regfile28_t),
260         .regfile29(regfile29_t),
261         .regfile30(regfile30_t),
262         .regfile31(regfile31_t)
263 );
264 DMEM dmem_inst(
265     .CS(CS_t),
266     .DM_R(DM_R_t),
267     .DM_W(DM_W_t),
268     .DMEMaddr(r_t),
269     .DMEMdata(DMEMdata_t),
270     .clk(clk_in),
271     .rt(rt_t)
272 );
273 IMEM imem_inst(
274     .IM_R(IM_R_t),
275     .address(pc_temp),
276     .func(func_t),

```

```
277     .index(index_t),
278 .instr(inst_t),    .imdtT(imdtT_t),
279     .op(op_t),
280     .rdc(rdc_t),
281     .rsc(rsc_t),
282     .rtc(rtc_t),
283     .shamtT(shamtT_t)
284 );
285
286
287 endmodule
```

装

订

线